Easy Zig

JW Lee

Easy Zig

A Comprehensive Beginner's Guide to the Zig Programming Language

by JW Lee

Preface

Welcome to **Easy Zig**! This book is designed for beginners who want to learn the Zig programming language from scratch. Whether you're coming from Python, JavaScript, C, or this is your first programming language, this book will guide you step by step.

What is Zig?

Zig is a modern systems programming language created by Andrew Kelley in 2016. It's designed to be a "better C"- providing low-level control over memory and hardware while avoiding many of the pitfalls that make C and C++ difficult and error-prone.

```
┌──────────────────────────────────────────────────────────┐
│                  Programming Language Spectrum            │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  High-Level                                   Low-Level  │
│  (Easier)                                     (Faster)   │
│                                                          │
│  Python ─── JavaScript ─── Go ─── Zig ─── C ─── ASM      │
│     │           │           │        │       │       │   │
│  Scripting    Web Apps    Services  Systems  OS    Hardware │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Why Choose Zig?

1. Simplicity

Unlike C++, Zig has no hidden control flow, no hidden memory allocations, and no operator overloading. What you see is what you get.

2. Safety Without Garbage Collection

Zig catches many bugs at compile time. It has no null pointers by default, bounds checking on arrays, and explicit error handling.

3. Performance

Zig compiles to highly optimized machine code, comparable to C and C++. It's used in production by companies like Uber.

4. C Interoperability

Zig can directly import and use C header files. No bindings needed. This means you have access to decades of C libraries.

5. Comptime (Compile-Time Execution)

Zig can run code at compile time, enabling powerful metaprogramming without macros or templates.

```
┌──────────────────────────────────────────────────────────┐
│                    Zig's Key Features                    │
├──────────────────────────────────────────────────────────┤
│                                                          │
│                                                          │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐ │
│   │  No GC       │   │ C Compat     │   │  Comptime    │ │
│   │ ──────────   │   │ ──────────   │   │ ──────────   │ │
│   │ Manual       │   │ Import .h    │   │ Run code     │ │
│   │ memory       │   │ files        │   │ at compile   │ │
│   │ control      │   │ directly     │   │ time         │ │
│   └──────────────┘   └──────────────┘   └──────────────┘ │
│                                                          │
│                                                          │
│   ┌──────────────┐   ┌──────────────┐   ┌──────────────┐ │
│   │ Safety       │   │  Fast        │   │ Cross-plat   │ │
│   │ ──────────   │   │ ──────────   │   │ ──────────   │ │
│   │ Compile-     │   │ LLVM         │   │ Compile      │ │
│   │ time         │   │ optimized    │   │ to any       │ │
│   │ checks       │   │ code         │   │ target       │ │
│   └──────────────┘   └──────────────┘   └──────────────┘ │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Setting Up Zig

Installation

macOS (Homebrew):

```
brew install zig
```

Linux:

```
# Download from ziglang.org and extract
wget https://ziglang.org/download/zig-linux-x86_64.tar.xz
```

```
tar -xf zig-linux-x86_64.tar.xz
export PATH=$PATH:./zig-linux-x86_64
```

Windows: Download the installer from https://ziglang.org/download/

Verify Installation

```
zig version
```

Your First Command

```
# Create a file called hello.zig, then run:
zig run hello.zig
```

How to Use This Book

Each chapter introduces one concept with: 1. **Explanation** - What the concept is and why it matters 2. **Diagrams** - Visual representations of how things work 3. **Code Examples** - Working code you can run yourself 4. **Output** - What you should see when running the code 5. **Key Points** - Summary of important takeaways

Let's begin your Zig journey!

Chapter 1: Hello World

Every programming journey starts with Hello World. This simple program teaches you the basic structure of a Zig program.

Understanding the Program Structure

```
┌──────────────────────────────────────────────────────────┐
│                    Zig Program Structure                 │
├──────────────────────────────────────────────────────────┤
│                                                          │
│    ┌──────────────────────────────────┐                  │
│    │  1. IMPORTS                      │                  │
│    │     const std = @import("std");  │                  │
│    └──────────────────────────────────┘                  │
│                      │                                   │
│                      ▼                                   │
│    ┌──────────────────────────────────┐                  │
│    │  2. CONSTANTS & GLOBALS          │                  │
│    │     const MAX = 100;             │                  │
│    └──────────────────────────────────┘                  │
│                      │                                   │
│                      ▼                                   │
│    ┌──────────────────────────────────┐                  │
│    │  3. FUNCTIONS                    │                  │
```

```
│    │     fn helper() { ... }            │                    │
│    └────────────────────────────────────┘                    │
│                      │                                       │
│                      ▼                                       │
│    ┌────────────────────────────────────┐                    │
│    │  4. MAIN FUNCTION (Entry Point)    │                    │
│    │     pub fn main() void { ... }     │                    │
│    └────────────────────────────────────┘                    │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Your First Program

```zig
// ============================================================
// Chapter 1: Hello World - Your First Zig Program
// ============================================================

// STEP 1: Import the standard library
// The @import function brings in external code
// "std" is Zig's standard library with useful functions
const std = @import("std");

// STEP 2: Define the main function
// - "pub" means public (can be called from outside)
// - "fn" declares a function
// - "main" is the entry point - where the program starts
// - "void" means this function returns nothing
pub fn main() void {
    // STEP 3: Print to the console
    // std.debug.print() outputs text to the terminal
    // \n creates a new line
    // .{} is an empty tuple for format arguments
    std.debug.print("Hello, World!\n", .{});
}
```

Output:

```
Hello, World!
```

Breaking Down Each Part

The Import Statement

```zig
const std = @import("std");
```

This line does three things: 1. `@import("std")` - Loads Zig's standard library 2. `const std` - Creates a constant named `std` 3. `=` - Assigns the library to the constant

Think of it like this:

```
┌──────────────────────────────────────────────────────────────┐
│                        @import("std")                        │
├──────────────────────────────────────────────────────────────┤
│                                                              │
```

```
│   Standard Library Contents:                                  │
│   ┌──────────────┐ ┌──────────────┐ ┌──────────────┐         │
│   │   debug      │ │    mem       │ │    fmt       │         │
│   │   .print()   │ │   .copy()    │ │  .format()   │         │
│   └──────────────┘ └──────────────┘ └──────────────┘         │
│                                                               │
│   ┌──────────────┐ ┌──────────────┐ ┌──────────────┐         │
│   │    heap      │ │    fs        │ │    os        │         │
│   │ .allocator()│ │ .openFile()│ │  .system()   │         │
│   └──────────────┘ └──────────────┘ └──────────────┘         │
│                                                               │
└───────────────────────────────────────────────────────────────┘
```

The Main Function

```zig
pub fn main() void {
    // code here
}
```

- `pub` - Makes the function visible to the runtime
- `fn` - Keyword to declare a function
- `main` - Special name recognized as the program's entry point
- `()` - Empty parentheses mean no parameters
- `void` - Return type (nothing returned)
- `{ }` - Curly braces contain the function body

The Print Statement

```zig
std.debug.print("Hello, World!\n", .{});
```

Let's break this down: - `std` - The standard library we imported - `.debug` - A namespace within std for debugging utilities - `.print()` - A function that outputs formatted text - `"Hello, World!\n"` - The format string (text to print) - `\n` - Escape sequence for a new line - `.{}` - An empty anonymous struct literal (tuple) for arguments

Printing with Variables

Now let's make our program more dynamic by using variables:

```zig
const std = @import("std");

pub fn main() void {
    // Declare a string constant
    // "Zig" is automatically typed as a pointer to bytes
    const name = "Zig";

    // Declare an integer constant
    // : u32 explicitly specifies the type as unsigned 32-bit
    const year: u32 = 2016;

    // Print with format specifiers:
    // {s} - format as string
    // {d} - format as decimal number
    std.debug.print("Welcome to {s}!\n", .{name});
```

```zig
    std.debug.print("{s} was created in {d}.\n", .{ name, year });

    // Multiple values in the tuple
    const version = "0.11";
    const is_stable = true;
    std.debug.print("Version: {s}, Stable: {}\n", .{ version, is_stable });
}
```

Output:

```
Welcome to Zig!
Zig was created in 2016.
Version: 0.11, Stable: true
```

Format Specifiers Reference

Format Specifiers	
Specifier	**Description**
{s}	String
{d}	Decimal number (integer)
{x}	Hexadecimal (lowercase)
{X}	Hexadecimal (uppercase)
{b}	Binary
{c}	Character
{}	Default format (auto-detect)
{any}	Print any type (for debugging)
{?}	Optional value
{e}	Scientific notation
Modifiers	
{d:5}	Minimum width of 5
{d:0>5}	Right-align, pad with zeros, width 5
{d:<5}	Left-align, width 5
{d:.2}	2 decimal places (for floats)

Advanced Printing Examples

```zig
const std = @import("std");

pub fn main() void {
    // Number formatting
    const num: i32 = 42;
    const float: f64 = 3.14159265359;
    const byte: u8 = 255;

    std.debug.print("Decimal:     {d}\n", .{num});
```

```
    std.debug.print("Hexadecimal: 0x{x}\n", .{byte});      // 0xff
    std.debug.print("Binary:      0b{b}\n", .{byte});      // 0b11111111
    std.debug.print("Float:       {d:.2}\n", .{float});    // 3.14
    std.debug.print("Scientific:  {e}\n", .{float});       // 3.14159e+00

    // Padding and alignment
    std.debug.print("Right aligned: |{d:>10}|\n", .{num}); // |        42|
    std.debug.print("Left aligned:  |{d:<10}|\n", .{num}); // |42        |
    std.debug.print("Zero padded:   |{d:0>10}|\n", .{num});// |0000000042|
}
```

Output:

```
Decimal:     42
Hexadecimal: 0xff
Binary:      0b11111111
Float:       3.14
Scientific:  3.14159265359e0
Right aligned: |        42|
Left aligned:  |42        |
Zero padded:   |0000000042|
```

Key Points

1. **Every Zig program needs a `main` function** - This is where execution begins
2. **Import before use** - Use `@import` to bring in libraries
3. **Format strings are type-checked** - Zig verifies your format specifiers match your arguments at compile time
4. **Semicolons are required** - Unlike JavaScript, you can't skip them
5. **Comments use `//`** - Everything after `//` on a line is ignored

---

Chapter 2: Variables

Variables are containers that store values. Zig has a clear distinction between values that can change and values that cannot.

The Two Types of Variables

```
┌──────────────────────────────────────────────────────────────┐
│                     Variable Types in Zig                    │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   ┌──────────────────────┐      ┌──────────────────────┐    │
│   │       const          │      │        var           │    │
│   │    (Immutable)       │      │     (Mutable)        │    │
│   ├──────────────────────┤      ├──────────────────────┤    │
│   │                      │      │                      │    │
│   │  const x = 5;        │      │  var x = 5;          │    │
│   │  x = 10; // ERROR!   │      │  x = 10; // OK!      │    │
│   │                      │      │                      │    │
```

```
│   │  • Cannot change    │      │  • Can change       │      │
│   │  • Preferred        │      │  • Use when needed  │      │
│   │  • Thread-safe      │      │  • Be careful       │      │
│   │  • Optimizable      │      │                     │      │
│   └─────────────────────┘      └─────────────────────┘      │
│                                                             │
│   RULE: Always use const unless you need to modify!         │
│                                                             │
└─────────────────────────────────────────────────────────────┘
```

Constants (const)

Constants are values that never change after initialization. They're the default choice in Zig.

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // CONSTANTS - Values that never change
    // ========================================

    // Type is inferred (Zig figures it out)
    const greeting = "Hello";           // []const u8 (string)
    const answer = 42;                   // comptime_int
    const pi = 3.14159;                  // comptime_float

    // Type is explicit (you specify it)
    const age: u32 = 25;                 // Unsigned 32-bit integer
    const temperature: f64 = 98.6;       // 64-bit float
    const is_valid: bool = true;         // Boolean

    std.debug.print("greeting: {s}\n", .{greeting});
    std.debug.print("answer: {d}\n", .{answer});
    std.debug.print("pi: {d:.5}\n", .{pi});
    std.debug.print("age: {d}\n", .{age});
    std.debug.print("temperature: {d}\n", .{temperature});
    std.debug.print("is_valid: {}\n", .{is_valid});

    // This would cause a compile error:
    // age = 26;  // ERROR: cannot assign to constant
}
```

Output:

```
greeting: Hello
answer: 42
pi: 3.14159
age: 25
temperature: 98.6
is_valid: true
```

Variables (var)

Variables can be changed after initialization. Use them only when necessary.

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // VARIABLES - Values that can change
    // ========================================

    var counter: i32 = 0;
    std.debug.print("Initial counter: {d}\n", .{counter});

    // Modify the variable
    counter = counter + 1;  // Now it's 1
    std.debug.print("After +1: {d}\n", .{counter});

    counter += 10;          // Shorthand for counter = counter + 10
    std.debug.print("After +=10: {d}\n", .{counter});

    counter *= 2;           // Double it
    std.debug.print("After *=2: {d}\n", .{counter});

    // Variables must have a type when declared with var
    var name: []const u8 = "Alice";
    std.debug.print("Name: {s}\n", .{name});

    name = "Bob";           // Change the value
    std.debug.print("New name: {s}\n", .{name});
}
```

Output:

```
Initial counter: 0
After +1: 1
After +=10: 11
After *=2: 22
Name: Alice
New name: Bob
```

Memory Layout of Variables

```
┌──────────────────────────────────────────────────────────────┐
│                    Memory Layout                             │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   When you write: var x: i32 = 42;                           │
│                                                              │
│   Stack Memory:                                              │
│   ┌──────────────────────────────────┐                       │
│   │  Address    │  Value  │  Variable  │                    │
│   ├──────────────┼─────────┼────────────┤                    │
│   │  0x1000     │   42    │     x      │ ← 4 bytes (i32)    │
│   │  0x1004     │   ...   │  (next)    │                    │
│   └──────────────┴─────────┴────────────┘                    │
│                                                              │
```

```
│   Different types use different amounts of memory:                │
│   ┌──────────────────────────────────────────────────────┐        │
│   │ Type     │ Size     │ Memory Visualization           │        │
│   ├──────────┼──────────┼────────────────────────────────┤        │
│   │ u8       │ 1 byte   │ [  ]                           │        │
│   │ u16      │ 2 bytes  │ [    ]                         │        │
│   │ u32      │ 4 bytes  │ [        ]                     │        │
│   │ u64      │ 8 bytes  │ [                ]             │        │
│   │ bool     │ 1 byte   │ [  ]                           │        │
│   └──────────┴──────────┴────────────────────────────────┘        │
│                                                                    │
└────────────────────────────────────────────────────────────────────┘
```

Undefined Values

Sometimes you want to declare a variable but assign its value later. Use `undefined`:

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // UNDEFINED - Declare now, assign later
    // ========================================

    // Declare without initializing
    // WARNING: Using an undefined value before assignment is UB!
    var result: i32 = undefined;

    // ... some code that determines the value ...

    // Now assign a value
    result = calculateSomething();

    std.debug.print("Result: {d}\n", .{result});
}

fn calculateSomething() i32 {
    return 42;
}
```

Output:

```
Result: 42
```

```
┌──────────────────────────────────────────────────────────────────┐
│                       UNDEFINED WARNING                          │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│   ⚠  DANGER: Reading undefined values is undefined              │
│       behavior (UB). The program may:                            │
│                                                                  │
│       • Crash                                                    │
│       • Return garbage values                                    │
│       • Behave unpredictably                                     │
```

```
│       • Work in debug but fail in release                    │
│                                                               │
│   SAFE PATTERN:                                               │
│   ┌──────────────────────────────────────────┐                │
│   │  var x: i32 = undefined;                  │                │
│   │  x = computeValue();  // MUST assign first! │              │
│   │  use(x);              // Now safe to use   │              │
│   └──────────────────────────────────────────┘                │
│                                                               │
│   UNSAFE PATTERN (DON'T DO THIS):                             │
│   ┌──────────────────────────────────────────┐                │
│   │  var x: i32 = undefined;                  │                │
│   │  use(x);  // UB! x has garbage value      │                │
│   └──────────────────────────────────────────┘                │
│                                                               │
└───────────────────────────────────────────────────────────────┘
```

Block Expressions

Zig allows you to use blocks to compute complex initial values:

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // BLOCK EXPRESSIONS - Complex initialization
    // ========================================

    // Use a block to compute a value
    const result = blk: {
        var temp: i32 = 10;
        temp *= 2;           // 20
        temp += 5;           // 25
        temp = temp * temp;  // 625
        break :blk temp;     // Return the value
    };

    std.debug.print("Result: {d}\n", .{result});

    // Useful for conditional initialization
    const value: i32 = 42;
    const description = blk: {
        if (value < 0) {
            break :blk "negative";
        } else if (value == 0) {
            break :blk "zero";
        } else {
            break :blk "positive";
        }
    };

    std.debug.print("{d} is {s}\n", .{ value, description });
}
```

Output:

```
Result: 625
42 is positive
```

Compile-Time Constants

Constants prefixed with `comptime` or defined at the top level are evaluated at compile time:

```zig
const std = @import("std");

// ========================================
// TOP-LEVEL CONSTANTS (Compile-time)
// ========================================

// These are computed BEFORE the program runs
const SECONDS_PER_MINUTE = 60;
const MINUTES_PER_HOUR = 60;
const HOURS_PER_DAY = 24;

// Computed from other constants (still compile-time)
const SECONDS_PER_HOUR = SECONDS_PER_MINUTE * MINUTES_PER_HOUR;
const SECONDS_PER_DAY = SECONDS_PER_HOUR * HOURS_PER_DAY;

// Function executed at compile time
fn factorial(n: u64) u64 {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}

// Pre-computed factorials
const FACT_5 = factorial(5);   // Computed at compile time!
const FACT_10 = factorial(10); // Also compile time!

pub fn main() void {
    std.debug.print("Seconds per hour: {d}\n", .{SECONDS_PER_HOUR});
    std.debug.print("Seconds per day: {d}\n", .{SECONDS_PER_DAY});
    std.debug.print("5! = {d}\n", .{FACT_5});
    std.debug.print("10! = {d}\n", .{FACT_10});
}
```

Output:

```
Seconds per hour: 3600
Seconds per day: 86400
5! = 120
10! = 3628800
```

```
┌──────────────────────────────────────────────────────────────┐
│              Compile-Time vs Run-Time                        │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   SOURCE CODE                                                │
│        │                                                     │
│        ▼                                                     │
```

```
┌──────────────────────────────────────────────┐
│           COMPILE TIME                       │
│   • const at top level                       │
│   • comptime blocks                          │
│   • Type computations                        │
│   • Generic instantiation                    │
│                                              │
│   Example: const X = factorial(10);          │
│            Computed now! Not at runtime.     │
└──────────────────────────────────────────────┘
       │
       ▼
EXECUTABLE (X is already 3628800)
       │
       ▼
┌──────────────────────────────────────────────┐
│            RUN TIME                          │
│   • var variables                            │
│   • User input                               │
│   • Function calls with runtime values       │
│                                              │
│   Example: var x = getUserInput();           │
│            Computed now, during execution.   │
└──────────────────────────────────────────────┘
```

Naming Conventions

```
┌──────────────────────────────────────────────────────────────┐
│                    Naming Conventions                        │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   Type              │  Convention        │  Example          │
│   ──────────────────────────────────────────────────────     │
│   Local variables   │  snake_case        │  my_variable      │
│   Constants         │  snake_case or     │  max_size         │
│                     │  SCREAMING_CASE    │  MAX_SIZE         │
│   Functions         │  camelCase         │  calculateSum     │
│   Types/Structs     │  PascalCase        │  MyStruct         │
│   Compile-time      │  SCREAMING_CASE    │  BUFFER_SIZE      │
│                                                              │
│   Examples:                                                  │
│   ┌──────────────────────────────────────────────────┐       │
│   │  const BUFFER_SIZE = 1024;  // Compile-time const │       │
│   │  var user_count: u32 = 0;   // Mutable variable   │       │
│   │  const User = struct {};    // Type definition   │       │
│   │  fn getUserName() {}        // Function          │       │
│   └──────────────────────────────────────────────────┘       │
│                                                              │
```

```
└──────────────────────────────────────────────────────────────────┘
```

Key Points

1. **Prefer `const` over `var`** - Immutability prevents bugs
2. **Types can be inferred or explicit** - Zig figures out types when possible
3. **`undefined` is dangerous** - Only use when you'll assign before reading
4. **Block expressions** - Use labeled blocks for complex initialization
5. **Compile-time evaluation** - Top-level constants are computed before runtime

---

Chapter 3: Types

Zig is a statically typed language, meaning every value has a type known at compile time. Understanding types is fundamental to writing correct Zig code.

The Type System Overview

```
┌──────────────────────────────────────────────────────────────────┐
│                      Zig Type Hierarchy                          │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│                          All Types                               │
│                              │                                   │
│              ┌───────────────┴───────────────┐                   │
│              │                               │                   │
│       Primitive                          Composite               │
│              │                               │                   │
│     ┌────────┴────────┐                ┌────┴────┐              │
│     │                 │                │         │              │
│  Numeric        Other            Arrays    Structs    Unions     │
│     │                 │                │         │         │    │
│  ┌──┴──┐        ┌─────┴─────┐       Slices    Enums     Optionals│
│  │     │        │           │       Pointers                    │
│ Int Float  Bool        Void       Vectors                        │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Integer Types

Zig offers fine-grained control over integer sizes:

```
┌──────────────────────────────────────────────────────────────────┐
│                      Integer Types                               │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│   SIGNED (can be negative)          UNSIGNED (positive only)     │
│   ────────────────────────          ────────────────────────     │
│   i8   : -128 to 127                u8   : 0 to 255              │
│   i16  : -32768 to 32767            u16  : 0 to 65535            │
```

```
│   i32  : -2.1B to 2.1B              u32  : 0 to 4.2B        │
│   i64  : huge range                 u64  : 0 to huge        │
│   i128 : even huger                 u128 : even huger       │
│                                                             │
│   Special Types:                                            │
│   ────────────────                                          │
│   isize : signed pointer-size (32 or 64 bits)              │
│   usize : unsigned pointer-size (for array indices)         │
│                                                             │
│   Arbitrary Width:                                          │
│   ──────────────────                                        │
│   u1, u2, u3, ..., u65535  (any bit width!)                │
│   i1, i2, i3, ..., i65535                                   │
│                                                             │
└─────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // SIGNED INTEGERS (can be negative)
    // ========================================

    const tiny: i8 = -128;        // 8-bit: -128 to 127
    const small: i16 = -32768;    // 16-bit
    const medium: i32 = -2147483648; // 32-bit
    const large: i64 = -9223372036854775808; // 64-bit

    std.debug.print("i8 min:  {d}\n", .{tiny});
    std.debug.print("i16 min: {d}\n", .{small});
    std.debug.print("i32 min: {d}\n", .{medium});
    std.debug.print("i64 min: {d}\n", .{large});

    // ========================================
    // UNSIGNED INTEGERS (only positive)
    // ========================================

    const byte: u8 = 255;          // 8-bit: 0 to 255
    const word: u16 = 65535;       // 16-bit
    const dword: u32 = 4294967295; // 32-bit
    const qword: u64 = 18446744073709551615; // 64-bit

    std.debug.print("\nu8 max:  {d}\n", .{byte});
    std.debug.print("u16 max: {d}\n", .{word});
    std.debug.print("u32 max: {d}\n", .{dword});
    std.debug.print("u64 max: {d}\n", .{qword});

    // ========================================
    // ARBITRARY BIT-WIDTH
    // ========================================

    const nibble: u4 = 15;         // 4-bit: 0 to 15
    const six_bits: u6 = 63;       // 6-bit: 0 to 63
```

```zig
    const twelve: u12 = 4095;      // 12-bit: 0 to 4095

    std.debug.print("\nu4 max:  {d}\n", .{nibble});
    std.debug.print("u6 max:  {d}\n", .{six_bits});
    std.debug.print("u12 max: {d}\n", .{twelve});
}
```

Output:

```
i8 min:  -128
i16 min: -32768
i32 min: -2147483648
i64 min: -9223372036854775808

u8 max:  255
u16 max: 65535
u32 max: 4294967295
u64 max: 18446744073709551615

u4 max:  15
u6 max:  63
u12 max: 4095
```

Floating Point Types

```
┌──────────────────────────────────────────────────────────────┐
│                    Floating Point Types                      │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   Type    Size      Precision        Range (approx)          │
│   ────    ────      ─────────        ──────────────          │
│   f16     16-bit    ~3 digits        ±65504                  │
│   f32     32-bit    ~7 digits        ±3.4 × 10^38            │
│   f64     64-bit    ~15 digits       ±1.8 × 10^308           │
│   f128    128-bit   ~33 digits       ±1.2 × 10^4932          │
│                                                              │
│   Memory Layout (f32 - IEEE 754):                            │
│    ┌───┬──────────┬──────────────────────┐                   │
│    │ S │ Exponent │       Mantissa       │                   │
│    │ 1 │  8 bits  │       23 bits        │                   │
│    └───┴──────────┴──────────────────────┘                   │
│                                                              │
│   Example: 3.14159 in f32                                    │
│   Sign: 0 (positive)                                         │
│   Exponent: 10000000 (1)                                     │
│   Mantissa: 10010010000111111010000                          │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
```

```zig
    // ========================================
    // FLOATING POINT NUMBERS
    // ========================================

    const pi_32: f32 = 3.14159265358979;  // Less precise
    const pi_64: f64 = 3.14159265358979;  // More precise

    std.debug.print("f32 pi: {d:.10}\n", .{pi_32});
    std.debug.print("f64 pi: {d:.14}\n", .{pi_64});

    // Scientific notation
    const avogadro: f64 = 6.022e23;  // 6.022 × 10^23
    const planck: f64 = 6.626e-34;  // 6.626 × 10^-34

    std.debug.print("Avogadro: {e}\n", .{avogadro});
    std.debug.print("Planck: {e}\n", .{planck});

    // Special values
    const inf = std.math.inf(f64);      // Infinity
    const neg_inf = -std.math.inf(f64); // Negative infinity
    const nan = std.math.nan(f64);      // Not a Number

    std.debug.print("Infinity: {d}\n", .{inf});
    std.debug.print("NaN: {d}\n", .{nan});

    // Precision matters!
    const a: f32 = 0.1;
    const b: f32 = 0.2;
    const sum: f32 = a + b;
    std.debug.print("\n0.1 + 0.2 = {d:.20}\n", .{sum});
    std.debug.print("(Not exactly 0.3 due to binary representation)\n", .{});
}
```

Output:

```
f32 pi: 3.1415927410
f64 pi: 3.14159265358979
Avogadro: 6.022e23
Planck: 6.626e-34
Infinity: inf
NaN: nan

0.1 + 0.2 = 0.30000001192092895508
(Not exactly 0.3 due to binary representation)
```

Boolean Type

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // BOOLEAN - true or false
    // ========================================
```

```zig
    const is_active: bool = true;
    const is_empty: bool = false;

    std.debug.print("is_active: {}\n", .{is_active});
    std.debug.print("is_empty: {}\n", .{is_empty});

    // Boolean from comparison
    const x: i32 = 10;
    const y: i32 = 20;

    const is_equal = x == y;     // false
    const is_less = x < y;       // true
    const is_greater = x > y;    // false

    std.debug.print("\n{d} == {d}: {}\n", .{ x, y, is_equal });
    std.debug.print("{d} < {d}: {}\n", .{ x, y, is_less });
    std.debug.print("{d} > {d}: {}\n", .{ x, y, is_greater });

    // Boolean in memory is 1 byte
    std.debug.print("\nSize of bool: {d} byte\n", .{@sizeOf(bool)});
}
```

Output:

```
is_active: true
is_empty: false

10 == 20: false
10 < 20: true
10 > 20: false

Size of bool: 1 byte
```

Type Casting and Conversion

```
┌──────────────────────────────────────────────────────────────────┐
│                      Type Conversions                            │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│   WIDENING (Safe - Automatic)                                    │
│   ──────────────────────────────                                 │
│   u8 ──────► u16 ──────► u32 ──────► u64                        │
│        OK         OK          OK                                 │
│                                                                  │
│   i8 ──────► i16 ──────► i32 ──────► i64                        │
│        OK         OK          OK                                 │
│                                                                  │
│   NARROWING (Dangerous - Requires @intCast)                      │
│   ──────────────────────────────────────────                     │
│   u32 ──────► u16 ──────► u8                                    │
│        ⚠          ⚠        May lose data!                        │
│                                                                  │
```

```
│   CONVERSION FUNCTIONS:                                          │
│   ────────────────────                                           │
│   @intCast      : Convert between integer types                  │
│   @floatCast    : Convert between float types                    │
│   @intFromFloat : Float to integer (truncates)                   │
│   @floatFromInt : Integer to float                               │
│   @truncate     : Force truncation                               │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // WIDENING (Safe, implicit)
    // ========================================

    const small: u8 = 42;
    const medium: u16 = small;   // u8 fits in u16, OK
    const large: u32 = medium;   // u16 fits in u32, OK

    std.debug.print("Widening: {d} -> {d} -> {d}\n", .{ small, medium, large });

    // ========================================
    // NARROWING (Dangerous, requires @intCast)
    // ========================================

    const big: u32 = 200;
    const tiny: u8 = @intCast(big);  // You must be sure it fits!

    std.debug.print("Narrowing: {d} -> {d}\n", .{ big, tiny });

    // WARNING: This would crash if big > 255
    // const too_big: u32 = 300;
    // const crash: u8 = @intCast(too_big);  // Runtime error!

    // ========================================
    // FLOAT CONVERSIONS
    // ========================================

    const float64: f64 = 3.14159265358979;
    const float32: f32 = @floatCast(float64);  // Loses precision

    std.debug.print("\nf64: {d:.15}\n", .{float64});
    std.debug.print("f32: {d:.15}\n", .{float32});

    // Float to integer (truncates toward zero)
    const pi: f64 = 3.99999;
    const pi_int: i32 = @intFromFloat(pi);  // Becomes 3

    std.debug.print("\nFloat {d:.5} to int: {d}\n", .{ pi, pi_int });

    // Integer to float
    const count: i32 = 42;
```

```zig
    const count_float: f64 = @floatFromInt(count);

    std.debug.print("Int {d} to float: {d}\n", .{ count, count_float });

    // ========================================
    // SIGNED/UNSIGNED CONVERSION
    // ========================================

    const positive: u32 = 100;
    const signed: i32 = @intCast(positive);  // Safe if value fits

    std.debug.print("\nu32 {d} to i32: {d}\n", .{ positive, signed });
}
```

Output:

```
Widening: 42 -> 42 -> 42
Narrowing: 200 -> 200

f64: 3.141592653589790
f32: 3.141592741012573

Float 3.99999 to int: 3
Int 42 to float: 42

u32 100 to i32: 100
```

Type Information with @typeInfo

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
    // TYPE INTROSPECTION
    // ========================================

    // Get type name
    std.debug.print("Type names:\n", .{});
    std.debug.print("  i32: {s}\n", .{@typeName(i32)});
    std.debug.print("  []const u8: {s}\n", .{@typeName([]const u8)});
    std.debug.print("  bool: {s}\n", .{@typeName(bool)});

    // Get size of types
    std.debug.print("\nType sizes:\n", .{});
    std.debug.print("  i8:   {d} bytes\n", .{@sizeOf(i8)});
    std.debug.print("  i32:  {d} bytes\n", .{@sizeOf(i32)});
    std.debug.print("  i64:  {d} bytes\n", .{@sizeOf(i64)});
    std.debug.print("  f32:  {d} bytes\n", .{@sizeOf(f32)});
    std.debug.print("  f64:  {d} bytes\n", .{@sizeOf(f64)});
    std.debug.print("  bool: {d} byte\n", .{@sizeOf(bool)});

    // Get alignment
    std.debug.print("\nType alignments:\n", .{});
```

```
    std.debug.print("  i32: {d}-byte aligned\n", .{@alignOf(i32)});
    std.debug.print("  i64: {d}-byte aligned\n", .{@alignOf(i64)});
}
```

Output:

```
Type names:
  i32: i32
  []const u8: []const u8
  bool: bool

Type sizes:
  i8:   1 bytes
  i32:  4 bytes
  i64:  8 bytes
  f32:  4 bytes
  f64:  8 bytes
  bool: 1 byte

Type alignments:
  i32: 4-byte aligned
  i64: 8-byte aligned
```

Key Points

1. **Choose the right size** - Don't use i64 when i8 suffices
2. **Signed vs unsigned** - Use unsigned for counts/indices, signed for arithmetic
3. **Float precision** - f64 is more precise but uses more memory
4. **Explicit casting** - Zig requires you to explicitly cast when narrowing
5. **Comptime types** - Numeric literals without types are `comptime_int` or `comptime_float`

Chapter 4: Operators

Operators are symbols that perform operations on values. Zig provides a comprehensive set of operators for arithmetic, comparison, logic, and bit manipulation.

Operator Categories

```
┌──────────────────────────────────────────────────────────────┐
│                    Operator Categories                       │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   ARITHMETIC          COMPARISON         LOGICAL             │
│   ──────────          ──────────         ───────             │
│   +  Addition         ==  Equal          and  Logical AND    │
│   -  Subtraction      !=  Not equal      or   Logical OR     │
│   *  Multiplication   <   Less than      !    Logical NOT    │
│   /  Division         >   Greater than                        │
│   %  Modulo           <=  Less or equal                       │
│   -  Negation         >=  Greater/equal                       │
```

```
│                                                          │
│   BITWISE             ASSIGNMENT         SPECIAL           │
│   ───────             ──────────         ───────           │
│   &   AND             =   Assign         ++  (Not in Zig!) │
│   |   OR              +=  Add assign     --  (Not in Zig!) │
│   ^   XOR             -=  Sub assign     ?:  (Use if-else) │
│   ~   NOT             *=  Mul assign                       │
│   <<  Left shift      /=  Div assign                       │
│   >>  Right shift     %=  Mod assign                       │
│                                                            │
└────────────────────────────────────────────────────────────┘
```

Arithmetic Operators

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== ARITHMETIC OPERATORS ===\n\n", .{});

    const a: i32 = 17;
    const b: i32 = 5;

    // Basic operations
    std.debug.print("{d} + {d} = {d}\n", .{ a, b, a + b });   // 22
    std.debug.print("{d} - {d} = {d}\n", .{ a, b, a - b });   // 12
    std.debug.print("{d} * {d} = {d}\n", .{ a, b, a * b });   // 85

    // Division in Zig requires explicit functions
    // @divTrunc - truncates toward zero
    // @divFloor - floors toward negative infinity
    std.debug.print("{d} / {d} = {d} (truncated)\n", .{ a, b, @divTrunc(a, b) });

    // Modulo
    std.debug.print("{d} %% {d} = {d}\n", .{ a, b, @mod(a, b) });

    // Negation
    std.debug.print("-{d} = {d}\n", .{ a, -a });

    // ========================================
    // OVERFLOW-SAFE OPERATORS
    // ========================================

    std.debug.print("\n=== OVERFLOW HANDLING ===\n\n", .{});

    // Wrapping operators (wrap on overflow)
    var x: u8 = 250;
    x +%= 10;  // Wraps: 250 + 10 = 260 -> 4 (260 % 256)
    std.debug.print("250 +%% 10 = {d} (wrapped)\n", .{x});

    // Saturating operators (clamp at max/min)
    var y: u8 = 250;
    y +|= 10;  // Saturates: 250 + 10 = 255 (max u8)
```

```
    std.debug.print("250 +| 10 = {d} (saturated)\n", .{y});
}
```

Output:

```
=== ARITHMETIC OPERATORS ===

17 + 5 = 22
17 - 5 = 12
17 * 5 = 85
17 / 5 = 3 (truncated)
17 % 5 = 2
-17 = -17

=== OVERFLOW HANDLING ===

250 +% 10 = 4 (wrapped)
250 +| 10 = 255 (saturated)
```

Overflow Handling Visual

```
┌──────────────────────────────────────────────────────────────┐
│                      Overflow Handling                       │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   u8 range: 0 ──────────────────────────────────────► 255    │
│                                                              │
│   Normal (+): 250 + 10 = ?                                   │
│    ┌──────────────────────────────────────────┐              │
│    │  In debug: PANIC! Overflow detected!     │              │
│    │  In release: Undefined behavior (bad!)   │              │
│    └──────────────────────────────────────────┘              │
│                                                              │
│   Wrapping (+%): 250 +% 10 = 4                               │
│    ┌──────────────────────────────────────────┐              │
│    │  250 + 10 = 260                          │              │
│    │  260 mod 256 = 4                         │              │
│    │  Like a clock wrapping around            │              │
│    │                                          │              │
│    │  0 ←──────────────────────────────── 255 │              │
│    │  ↑                                     │ │              │
│    │  └──────────────── wraps ──────────────┘ │              │
│    └──────────────────────────────────────────┘              │
│                                                              │
│   Saturating (+|): 250 +| 10 = 255                           │
│    ┌──────────────────────────────────────────┐              │
│    │  250 + 10 = 260                          │              │
│    │  260 > 255, so clamp to 255              │              │
│    │  Like a cup that can't overflow          │              │
│    │                                          │              │
│    │  Result: ═══════════════════════════▌ 255 │              │
│                                                              │
```

```
│   │          (stuck at maximum)                    │       │
│   └──────────────────────────────────────────────┘       │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Comparison Operators

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== COMPARISON OPERATORS ===\n\n", .{});

    const x: i32 = 10;
    const y: i32 = 20;
    const z: i32 = 10;

    // Equality
    std.debug.print("{d} == {d}: {}\n", .{ x, y, x == y });   // false
    std.debug.print("{d} == {d}: {}\n", .{ x, z, x == z });   // true
    std.debug.print("{d} != {d}: {}\n", .{ x, y, x != y });   // true

    // Relational
    std.debug.print("\n{d} <  {d}: {}\n", .{ x, y, x < y });  // true
    std.debug.print("{d} >  {d}: {}\n", .{ x, y, x > y });    // false
    std.debug.print("{d} <= {d}: {}\n", .{ x, z, x <= z });   // true
    std.debug.print("{d} >= {d}: {}\n", .{ x, y, x >= y });   // false

    // Comparison results are booleans
    const is_equal: bool = x == z;
    const is_greater: bool = x > y;

    std.debug.print("\nStored results: is_equal={}, is_greater={}\n", .{ is_equal, is_greater }
}
```

Output:

```
=== COMPARISON OPERATORS ===

10 == 20: false
10 == 10: true
10 != 20: true

10 <  20: true
10 >  20: false
10 <= 10: true
10 >= 20: false

Stored results: is_equal=true, is_greater=false
```

Logical Operators

```
┌──────────────────────────────────────────────────────────┐
│                     Logical Operators                    │
```

```
┌──────────────────────────────────────────────────────────────┐
│                                                              │
│   AND (and): Both must be true                               │
│   ┌──────────────────────────────────────────────┐           │
│   │  A      │  B      │  A and B                 │           │
│   ├─────────┼─────────┼──────────────────────────┤           │
│   │ false   │ false   │   false                  │           │
│   │ false   │ true    │   false                  │           │
│   │ true    │ false   │   false                  │           │
│   │ true    │ true    │   true   ✓               │           │
│   └──────────────────────────────────────────────┘           │
│                                                              │
│   OR (or): At least one must be true                         │
│   ┌──────────────────────────────────────────────┐           │
│   │  A      │  B      │  A or B                  │           │
│   ├─────────┼─────────┼──────────────────────────┤           │
│   │ false   │ false   │   false                  │           │
│   │ false   │ true    │   true   ✓               │           │
│   │ true    │ false   │   true   ✓               │           │
│   │ true    │ true    │   true   ✓               │           │
│   └──────────────────────────────────────────────┘           │
│                                                              │
│   NOT (!): Inverts the value                                 │
│   ┌──────────────────────────────────────────────┐           │
│   │  A      │  !A                                │           │
│   ├─────────┼────────────────────────────────────┤           │
│   │ false   │  true                              │           │
│   │ true    │  false                             │           │
│   └──────────────────────────────────────────────┘           │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== LOGICAL OPERATORS ===\n\n", .{});

    const t = true;
    const f = false;

    // AND - both must be true
    std.debug.print("true  and true  = {}\n", .{t and t});
    std.debug.print("true  and false = {}\n", .{t and f});
    std.debug.print("false and true  = {}\n", .{f and t});
    std.debug.print("false and false = {}\n", .{f and f});

    // OR - at least one must be true
    std.debug.print("\ntrue  or true  = {}\n", .{t or t});
    std.debug.print("true  or false = {}\n", .{t or f});
    std.debug.print("false or true  = {}\n", .{f or t});
    std.debug.print("false or false = {}\n", .{f or f});
```

```
    // NOT - inverts
    std.debug.print("\nnot true  = {}\n", .{!t});
    std.debug.print("not false = {}\n", .{!f});

    // ========================================
    // SHORT-CIRCUIT EVALUATION
    // ========================================

    std.debug.print("\n=== SHORT-CIRCUIT EVALUATION ===\n\n", .{});

    // With 'and', if first is false, second is NOT evaluated
    // With 'or', if first is true, second is NOT evaluated

    const x: i32 = 0;

    // Safe division check using short-circuit
    const safe = x != 0 and @divTrunc(10, x) > 0;
    std.debug.print("Division check (x=0): {}\n", .{safe});
    // The division never happens because x != 0 is false!
}
```

Output:

```
=== LOGICAL OPERATORS ===

true  and true  = true
true  and false = false
false and true  = false
false and false = false

true  or true  = true
true  or false = true
false or true  = true
false or false = false

not true  = false
not false = true

=== SHORT-CIRCUIT EVALUATION ===

Division check (x=0): false
```

Bitwise Operators

```
┌──────────────────────────────────────────────────────────────┐
│                    Bitwise Operations                        │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   Given:  A = 0b11001010  (202)                              │
│           B = 0b10101100  (172)                              │
│                                                              │
│   AND (&): Both bits must be 1                               │
│    ┌──────────────────────────────────────────────┐          │
```

```
│   │  A:     1 1 0 0 1 0 1 0                    │
│   │  B:     1 0 1 0 1 1 0 0                    │
│   │         ─────────────────                  │
│   │  A & B: 1 0 0 0 1 0 0 0  = 136             │
│   └────────────────────────────────────────────┘
│
│   OR (|): At least one bit must be 1
│   ┌────────────────────────────────────────────┐
│   │  A:     1 1 0 0 1 0 1 0                    │
│   │  B:     1 0 1 0 1 1 0 0                    │
│   │         ─────────────────                  │
│   │  A | B: 1 1 1 0 1 1 1 0  = 238             │
│   └────────────────────────────────────────────┘
│
│   XOR (^): Exactly one bit must be 1
│   ┌────────────────────────────────────────────┐
│   │  A:     1 1 0 0 1 0 1 0                    │
│   │  B:     1 0 1 0 1 1 0 0                    │
│   │         ─────────────────                  │
│   │  A ^ B: 0 1 1 0 0 1 1 0  = 102             │
│   └────────────────────────────────────────────┘
│
│   NOT (~): Flip all bits
│   ┌────────────────────────────────────────────┐
│   │  A:     1 1 0 0 1 0 1 0                    │
│   │         ─────────────────                  │
│   │  ~A:    0 0 1 1 0 1 0 1  = 53              │
│   └────────────────────────────────────────────┘
│
│   LEFT SHIFT (<<): Shift bits left, fill with 0
│   ┌────────────────────────────────────────────┐
│   │  X:      0 0 0 0 1 1 1 1  = 15             │
│   │  X << 2: 0 0 1 1 1 1 0 0  = 60             │
│   │          (multiply by 2^n)                 │
│   └────────────────────────────────────────────┘
│
│   RIGHT SHIFT (>>): Shift bits right
│   ┌────────────────────────────────────────────┐
│   │  X:      0 0 0 0 1 1 1 1  = 15             │
│   │  X >> 2: 0 0 0 0 0 0 1 1  = 3              │
│   │          (divide by 2^n)                   │
│   └────────────────────────────────────────────┘
│
└──────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== BITWISE OPERATORS ===\n\n", .{});
```

```
    const a: u8 = 0b11001010;  // 202
    const b: u8 = 0b10101100;  // 172

    std.debug.print("a     = 0b{b:0>8} ({d})\n", .{ a, a });
    std.debug.print("b     = 0b{b:0>8} ({d})\n", .{ b, b });

    std.debug.print("\na & b = 0b{b:0>8} ({d}) - AND\n", .{ a & b, a & b });
    std.debug.print("a | b = 0b{b:0>8} ({d}) - OR\n", .{ a | b, a | b });
    std.debug.print("a ^ b = 0b{b:0>8} ({d}) - XOR\n", .{ a ^ b, a ^ b });
    std.debug.print("~a    = 0b{b:0>8} ({d}) - NOT\n", .{ ~a, ~a });

    // Bit shifts
    const x: u8 = 0b00001111;  // 15
    std.debug.print("\nx      = 0b{b:0>8} ({d})\n", .{ x, x });
    std.debug.print("x << 2 = 0b{b:0>8} ({d}) - Left shift\n", .{ x << 2, x << 2 });
    std.debug.print("x >> 2 = 0b{b:0>8} ({d}) - Right shift\n", .{ x >> 2, x >> 2 });

    // Practical example: Extracting bits
    std.debug.print("\n=== PRACTICAL: Bit Flags ===\n\n", .{});

    const READ: u8 = 0b00000001;   // Bit 0
    const WRITE: u8 = 0b00000010;  // Bit 1
    const EXECUTE: u8 = 0b00000100; // Bit 2

    var permissions: u8 = 0;

    // Set permissions
    permissions |= READ;      // Add read
    permissions |= WRITE;     // Add write
    std.debug.print("After adding READ|WRITE: 0b{b:0>8}\n", .{permissions});

    // Check permissions
    const can_read = (permissions & READ) != 0;
    const can_exec = (permissions & EXECUTE) != 0;
    std.debug.print("Can read: {}, Can execute: {}\n", .{ can_read, can_exec });

    // Remove permission
    permissions &= ~WRITE;    // Remove write
    std.debug.print("After removing WRITE: 0b{b:0>8}\n", .{permissions});
}
```

Output:

```
=== BITWISE OPERATORS ===

a     = 0b11001010 (202)
b     = 0b10101100 (172)

a & b = 0b10001000 (136) - AND
a | b = 0b11101110 (238) - OR
a ^ b = 0b01100110 (102) - XOR
~a    = 0b00110101 (53) - NOT

x      = 0b00001111 (15)
```

```
x << 2 = 0b00111100 (60) - Left shift
x >> 2 = 0b00000011 (3) - Right shift

=== PRACTICAL: Bit Flags ===

After adding READ|WRITE: 0b00000011
Can read: true, Can execute: false
After removing WRITE: 0b00000001
```

Key Points

1. **No ++ or –operators** - Use += 1 and –= 1 instead
2. **Explicit overflow handling** - Use +%, –%, *% for wrapping, +|, –|, *| for saturating
3. **Short-circuit evaluation** - `and` and `or` don't evaluate the second operand if not needed
4. **Bitwise operators** - Powerful for flags, masks, and low-level programming
5. **Division functions** - Use `@divTrunc`, `@divFloor`, `@mod`, `@rem` for explicit behavior

---

Chapter 5: Arrays

Arrays are fixed-size, contiguous collections of elements of the same type. They're one of the most fundamental data structures in Zig.

Array Memory Layout

```
┌──────────────────────────────────────────────────────────────┐
│                    Array in Memory                           │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   Declaration: const arr: [5]i32 = [5]i32{ 10, 20, 30, 40, 50 };
│                                                              │
│   Memory Layout (assuming 4-byte i32):                       │
│                                                              │
│   Address:  0x100   0x104   0x108   0x10C   0x110            │
│             ┌───────┬───────┬───────┬───────┬───────┐        │
│   Values:   │  10   │  20   │  30   │  40   │  50   │        │
│             └───────┴───────┴───────┴───────┴───────┘        │
│   Index:      [0]     [1]     [2]     [3]     [4]            │
│                                                              │
│   Key Properties:                                            │
│   • Size is fixed at compile time                            │
│   • Elements are contiguous (no gaps)                        │
│   • Access by index is O(1)                                  │
│   • Total size = element_size × count = 4 × 5 = 20 bytes     │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Creating Arrays

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== CREATING ARRAYS ===\n\n", .{});

    // ========================================
    // Method 1: Explicit type and values
    // ========================================
    const explicit: [5]i32 = [5]i32{ 10, 20, 30, 40, 50 };
    std.debug.print("Explicit: {any}\n", .{explicit});

    // ========================================
    // Method 2: Inferred length with [_]
    // ========================================
    const inferred = [_]i32{ 1, 2, 3, 4 };  // Compiler counts: 4 elements
    std.debug.print("Inferred: {any}\n", .{inferred});
    std.debug.print("Length: {d}\n", .{inferred.len});

    // ========================================
    // Method 3: Fill with same value
    // ========================================
    const zeros = [_]i32{0} ** 5;    // [0, 0, 0, 0, 0]
    const ones = [_]i32{1} ** 3;     // [1, 1, 1]
    const magic = [_]i32{42} ** 4;   // [42, 42, 42, 42]

    std.debug.print("Zeros: {any}\n", .{zeros});
    std.debug.print("Ones: {any}\n", .{ones});
    std.debug.print("Magic: {any}\n", .{magic});

    // ========================================
    // Method 4: Repeat a pattern
    // ========================================
    const pattern = [_]i32{ 1, 2 } ** 3;  // [1, 2, 1, 2, 1, 2]
    const wave = [_]i32{ 1, 2, 3, 2 } ** 2;

    std.debug.print("Pattern: {any}\n", .{pattern});
    std.debug.print("Wave: {any}\n", .{wave});

    // ========================================
    // Method 5: String arrays
    // ========================================
    const names = [_][]const u8{ "Alice", "Bob", "Charlie" };
    std.debug.print("Names: ", .{});
    for (names) |name| {
        std.debug.print("{s} ", .{name});
    }
    std.debug.print("\n", .{});
}
```

Output:

```
=== CREATING ARRAYS ===
```

```
Explicit: { 10, 20, 30, 40, 50 }
Inferred: { 1, 2, 3, 4 }
Length: 4
Zeros: { 0, 0, 0, 0, 0 }
Ones: { 1, 1, 1 }
Magic: { 42, 42, 42, 42 }
Pattern: { 1, 2, 1, 2, 1, 2 }
Wave: { 1, 2, 3, 2, 1, 2, 3, 2 }
Names: Alice Bob Charlie
```

Accessing and Modifying Arrays

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== ACCESSING ARRAYS ===\n\n", .{});

    const numbers = [_]i32{ 100, 200, 300, 400, 500 };

    // Access by index
    std.debug.print("First element (index 0): {d}\n", .{numbers[0]});
    std.debug.print("Third element (index 2): {d}\n", .{numbers[2]});
    std.debug.print("Last element (index {}): {d}\n", .{ numbers.len - 1, numbers[numbers.len -

    // ========================================
    // MODIFYING ARRAYS (must be var)
    // ========================================

    std.debug.print("\n=== MODIFYING ARRAYS ===\n\n", .{});

    var mutable = [_]i32{ 1, 2, 3, 4, 5 };
    std.debug.print("Before: {any}\n", .{mutable});

    // Modify individual elements
    mutable[0] = 100;
    mutable[4] = 500;
    std.debug.print("After mutable[0]=100, [4]=500: {any}\n", .{mutable});

    // Modify all elements in a loop
    for (&mutable) |*element| {
        element.* *= 2;
    }
    std.debug.print("After doubling all: {any}\n", .{mutable});
}
```

Output:

```
=== ACCESSING ARRAYS ===

First element (index 0): 100
Third element (index 2): 300
Last element (index 4): 500

=== MODIFYING ARRAYS ===
```

```
Before: { 1, 2, 3, 4, 5 }
After mutable[0]=100, [4]=500: { 100, 2, 3, 4, 500 }
After doubling all: { 200, 4, 6, 8, 1000 }
```

Iterating Over Arrays

```
┌──────────────────────────────────────────────────────────┐
│                    Array Iteration Methods                 │
├──────────────────────────────────────────────────────────┤
│                                                            │
│   1. Simple iteration (value only):                        │
│      for (array) |value| { ... }                           │
│                                                            │
│   2. With index:                                           │
│      for (array, 0..) |value, index| { ... }               │
│                                                            │
│   3. With pointer (to modify):                             │
│      for (&array) |*ptr| { ptr.* = newValue; }             │
│                                                            │
│   4. Multiple arrays (zip):                                │
│      for (arr1, arr2) |a, b| { ... }                       │
│                                                            │
└──────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== ARRAY ITERATION ===\n\n", .{});

    const numbers = [_]i32{ 10, 20, 30, 40, 50 };

    // ========================================
    // Method 1: Value only
    // ========================================
    std.debug.print("Values: ", .{});
    for (numbers) |value| {
        std.debug.print("{d} ", .{value});
    }
    std.debug.print("\n", .{});

    // ========================================
    // Method 2: With index
    // ========================================
    std.debug.print("\nWith index:\n", .{});
    for (numbers, 0..) |value, index| {
        std.debug.print("  [{d}] = {d}\n", .{ index, value });
    }

    // ========================================
    // Method 3: With pointer (modification)
    // ========================================
    var mutable = [_]i32{ 1, 2, 3, 4, 5 };
```

```zig
    for (&mutable) |*ptr| {
        ptr.* *= 10;  // Multiply each by 10
    }
    std.debug.print("\nAfter *10: {any}\n", .{mutable});

    // ========================================
    // Method 4: Multiple arrays (zip)
    // ========================================
    const a = [_]i32{ 1, 2, 3 };
    const b = [_]i32{ 10, 20, 30 };

    std.debug.print("\nZipped iteration:\n", .{});
    for (a, b, 0..) |x, y, i| {
        std.debug.print("  [{d}] a={d}, b={d}, sum={d}\n", .{ i, x, y, x + y });
    }
}
```

Output:

```
=== ARRAY ITERATION ===

Values: 10 20 30 40 50

With index:
  [0] = 10
  [1] = 20
  [2] = 30
  [3] = 40
  [4] = 50

After *10: { 10, 20, 30, 40, 50 }

Zipped iteration:
  [0] a=1, b=10, sum=11
  [1] a=2, b=20, sum=22
  [2] a=3, b=30, sum=33
```

Multidimensional Arrays

```
┌──────────────────────────────────────────────────────────┐
│                  2D Array (Matrix) Layout                 │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   Declaration: const matrix: [3][4]i32 = ...             │
│                                                          │
│   Logical View (3 rows, 4 columns):                      │
│                                                          │
│            Col 0   Col 1   Col 2   Col 3                 │
│          ┌────────┬────────┬────────┬────────┐           │
│   Row 0  │   1    │   2    │   3    │   4    │           │
│          ├────────┼────────┼────────┼────────┤           │
│   Row 1  │   5    │   6    │   7    │   8    │           │
│          ├────────┼────────┼────────┼────────┤           │
│
```

```
│   Row 2 │    9    │   10    │   11    │   12    │                    │
│         └─────────┴─────────┴─────────┴─────────┘                    │
│                                                                      │
│   Access: matrix[row][col]                                           │
│   Example: matrix[1][2] = 7                                          │
│                                                                      │
│   Memory Layout (row-major, contiguous):                             │
│   ┌─┬─┬─┬─┬─┬─┬─┬─┬─┬──┬──┬──┐                                      │
│   │1│2│3│4│5│6│7│8│9│10│11│12│                                      │
│   └─┴─┴─┴─┴─┴─┴─┴─┴─┴──┴──┴──┘                                      │
│   Row 0    Row 1      Row 2                                          │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== MULTIDIMENSIONAL ARRAYS ===\n\n", .{});

    // ========================================
    // 2D Array (Matrix)
    // ========================================
    const matrix = [3][4]i32{
        [_]i32{ 1, 2, 3, 4 },
        [_]i32{ 5, 6, 7, 8 },
        [_]i32{ 9, 10, 11, 12 },
    };

    // Access single element
    std.debug.print("matrix[1][2] = {d}\n", .{matrix[1][2]});  // 7

    // Print entire matrix
    std.debug.print("\nMatrix:\n", .{});
    for (matrix, 0..) |row, i| {
        std.debug.print("  Row {d}: ", .{i});
        for (row) |value| {
            std.debug.print("{d:4}", .{value});
        }
        std.debug.print("\n", .{});
    }

    // ========================================
    // 3D Array
    // ========================================
    const cube = [2][2][2]i32{
        [_][2]i32{
            [_]i32{ 1, 2 },
            [_]i32{ 3, 4 },
        },
        [_][2]i32{
            [_]i32{ 5, 6 },
            [_]i32{ 7, 8 },
        },
```

```
    };

    std.debug.print("\ncube[1][0][1] = {d}\n", .{cube[1][0][1]});  // 6
}
```

Output:

```
=== MULTIDIMENSIONAL ARRAYS ===

matrix[1][2] = 7

Matrix:
  Row 0:    1   2   3   4
  Row 1:    5   6   7   8
  Row 2:    9  10  11  12

cube[1][0][1] = 6
```

Array Operations

```
const std = @import("std");

pub fn main() void {
    std.debug.print("=== ARRAY OPERATIONS ===\n\n", .{});

    // ========================================
    // Concatenation (compile-time only)
    // ========================================
    const a = [_]i32{ 1, 2, 3 };
    const b = [_]i32{ 4, 5 };
    const combined = a ++ b;  // [1, 2, 3, 4, 5]

    std.debug.print("Concatenation: {any} ++ {any} = {any}\n", .{ a, b, combined });

    // ========================================
    // Multiplication (compile-time only)
    // ========================================
    const pattern = [_]i32{ 1, 2 };
    const repeated = pattern ** 4;  // [1, 2, 1, 2, 1, 2, 1, 2]

    std.debug.print("Repetition: {any} ** 4 = {any}\n", .{ pattern, repeated });

    // ========================================
    // Array Size Information
    // ========================================
    const arr: [5]i32 = [_]i32{ 10, 20, 30, 40, 50 };

    std.debug.print("\nArray info:\n", .{});
    std.debug.print("  Length: {d} elements\n", .{arr.len});
    std.debug.print("  Element size: {d} bytes\n", .{@sizeOf(i32)});
    std.debug.print("  Total size: {d} bytes\n", .{@sizeOf(@TypeOf(arr))});
}
```

Output:

```
=== ARRAY OPERATIONS ===

Concatenation: { 1, 2, 3 } ++ { 4, 5 } = { 1, 2, 3, 4, 5 }
Repetition: { 1, 2 } ** 4 = { 1, 2, 1, 2, 1, 2, 1, 2 }

Array info:
  Length: 5 elements
  Element size: 4 bytes
  Total size: 20 bytes
```

Key Points

1. **Fixed size** - Array length is known at compile time and cannot change
2. **Zero-indexed** - First element is at index 0
3. **Bounds checking** - Zig checks array bounds in debug mode
4. **Contiguous memory** - Elements are stored consecutively
5. **Use `[_]` for inferred length** - Compiler counts elements for you
6. **`++` and `**` are compile-time only** - Cannot concatenate arrays at runtime

---

Chapter 6: Slices

Slices are views into arrays or other contiguous memory. Unlike arrays, slices don't own the data - they just reference it. This makes them incredibly flexible for working with parts of arrays or for passing data to functions.

Slice vs Array

```
┌──────────────────────────────────────────────────────────────┐
│                    Array vs Slice                            │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   ARRAY: [5]i32                                              │
│   • Fixed size (5 elements)                                  │
│   • Size known at compile time                               │
│   • Owns its data                                            │
│   • Stored inline                                            │
│                                                              │
│   ┌─────┬─────┬─────┬─────┬─────┐                           │
│   │ 10  │ 20  │ 30  │ 40  │ 50  │  (Data inline)            │
│   └─────┴─────┴─────┴─────┴─────┘                           │
│                                                              │
│   SLICE: []i32                                               │
│   • Variable size                                            │
│   • Size known at runtime                                    │
│   • References data (doesn't own it)                         │
│   • Just a pointer + length                                  │
│                                                              │
│   ┌──────────────┐                                           │
│   │ ptr │ len=5  │──────┐                                    │
```

```
│   └──────────────────┘       │                              │
│                              ▼                              │
│   ┌─────┬─────┬─────┬─────┬─────┐                           │
│   │ 10  │ 20  │ 30  │ 40  │ 50  │  (Data elsewhere)         │
│   └─────┴─────┴─────┴─────┴─────┘                           │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Creating Slices

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== CREATING SLICES ===\n\n", .{});

    const array = [_]i32{ 10, 20, 30, 40, 50, 60, 70 };

    // ========================================
    // Method 1: Slice entire array
    // ========================================
    const all: []const i32 = &array;
    std.debug.print("Entire array: {any}\n", .{all});
    std.debug.print("Length: {d}\n", .{all.len});

    // ========================================
    // Method 2: Slice with range [start..end)
    // ========================================
    const middle = array[2..5];  // Elements at index 2, 3, 4
    std.debug.print("\nmiddle[2..5]: {any}\n", .{middle});

    // ========================================
    // Method 3: Open-ended slices
    // ========================================
    const from_start = array[0..3];   // First 3 elements
    const to_end = array[4..];        // From index 4 to end
    const everything = array[0..];    // Entire array

    std.debug.print("array[0..3]: {any}\n", .{from_start});
    std.debug.print("array[4..]:  {any}\n", .{to_end});
    std.debug.print("array[0..]: {any}\n", .{everything});
}
```

Output:

```
=== CREATING SLICES ===

Entire array: { 10, 20, 30, 40, 50, 60, 70 }
Length: 7

middle[2..5]: { 30, 40, 50 }
array[0..3]: { 10, 20, 30 }
array[4..]:  { 50, 60, 70 }
array[0..]: { 10, 20, 30, 40, 50, 60, 70 }
```

Slice Range Visualization

```
┌──────────────────────────────────────────────────────────────┐
│                      Slice Ranges                            │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   Original array: [10, 20, 30, 40, 50, 60, 70]              │
│   Indices:          0   1   2   3   4   5   6               │
│                                                              │
│   array[2..5]                                                │
│   ┌─────┬─────┬─────┬─────┬─────┬─────┬─────┐               │
│   │ 10  │ 20  │ 30  │ 40  │ 50  │ 60  │ 70  │               │
│   └─────┴─────┴─────┴─────┴─────┴─────┴─────┘               │
│                 ↑──────────────↑                             │
│                 2              5 (exclusive)                 │
│   Result: [30, 40, 50]                                       │
│                                                              │
│   array[0..3]  → [10, 20, 30]                               │
│   array[4..]   → [50, 60, 70]  (4 to end)                   │
│   array[..4]   → [10, 20, 30, 40]  (start to 4)             │
│                                                              │
│   NOTE: End index is EXCLUSIVE (not included)                │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Slices as Function Parameters

This is where slices really shine - they let you write functions that work with any size array:

```zig
const std = @import("std");

// This function works with ANY size array!
fn sum(values: []const i32) i32 {
    var total: i32 = 0;
    for (values) |v| {
        total += v;
    }
    return total;
}

fn printAll(data: []const u8) void {
    std.debug.print("Data: {s}\n", .{data});
}

fn average(values: []const i32) f64 {
    if (values.len == 0) return 0;
    const s = sum(values);
    return @as(f64, @floatFromInt(s)) / @as(f64, @floatFromInt(values.len));
}

pub fn main() void {
    std.debug.print("=== SLICES AS PARAMETERS ===\n\n", .{});
```

```zig
    // Different sized arrays
    const small = [_]i32{ 1, 2, 3 };
    const large = [_]i32{ 10, 20, 30, 40, 50 };

    // Same function works with both!
    std.debug.print("Sum of small: {d}\n", .{sum(&small)});
    std.debug.print("Sum of large: {d}\n", .{sum(&large)});

    // Even works with partial slices
    std.debug.print("Sum of large[1..4]: {d}\n", .{sum(large[1..4])});

    // Average
    std.debug.print("\nAverage of small: {d:.2}\n", .{average(&small)});
    std.debug.print("Average of large: {d:.2}\n", .{average(&large)});

    // String slice
    printAll("Hello, Zig!");
}
```

Output:

```
=== SLICES AS PARAMETERS ===

Sum of small: 6
Sum of large: 150
Sum of large[1..4]: 90

Average of small: 2.00
Average of large: 30.00
Data: Hello, Zig!
```

Key Points

1. **Slices don't own data** - They're views into existing memory
2. **Range syntax `[start..end]`** - End is exclusive
3. **Flexible function parameters** - Use `[]const T` to accept any size
4. **Length at runtime** - Unlike arrays, slice length can be computed at runtime
5. **Memory efficient** - Just a pointer + length (16 bytes on 64-bit)

Chapter 7: Pointers

Pointers store memory addresses. They're essential for systems programming, allowing you to reference data without copying it.

Pointer Basics

```
┌──────────────────────────────────────────────────────────────────┐
│                      Pointer Concept                             │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│   Variable x at address 0x1000:                                  │
```

```
│                                                                │
│    ┌──────────────┬──────────┬──────────┐                     │
│    │  Address     │  Value   │  Name    │                     │
│    ├──────────────┼──────────┼──────────┤                     │
│    │  0x1000      │   42     │   x      │                     │
│    └──────────────┴──────────┴──────────┘                     │
│                                                                │
│   Pointer ptr pointing to x:                                   │
│                                                                │
│    ┌──────────────┬──────────┬──────────┐                     │
│    │  Address     │  Value   │  Name    │                     │
│    ├──────────────┼──────────┼──────────┤                     │
│    │  0x2000      │ 0x1000   │   ptr    │──────┐              │
│    └──────────────┴──────────┴──────────┘      │              │
│                                                │              │
│                                                │              │
│                              ┌─────────────────┘              │
│                              ▼                                │
│    ┌──────────────┬──────────┬──────────┐                     │
│    │  0x1000      │   42     │   x      │                     │
│    └──────────────┴──────────┴──────────┘                     │
│                                                                │
│   ptr.*  means "the value at the address stored in ptr"        │
│   &x     means "the address of x"                              │
│                                                                │
└────────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== POINTER BASICS ===\n\n", .{});

    // Create a variable
    var value: i32 = 42;

    // Get a pointer to it with &
    const ptr: *i32 = &value;

    // Read through the pointer with .*
    std.debug.print("value = {d}\n", .{value});
    std.debug.print("ptr.* = {d}\n", .{ptr.*});

    // Modify through the pointer
    ptr.* = 100;
    std.debug.print("\nAfter ptr.* = 100:\n", .{});
    std.debug.print("value = {d}\n", .{value});
    std.debug.print("ptr.* = {d}\n", .{ptr.*});
}
```

Output:

```
=== POINTER BASICS ===

value = 42
```

```
ptr.* = 42

After ptr.* = 100:
value = 100
ptr.* = 100
```

Pointer Types

```
┌──────────────────────────────────────────────────────────┐
│                      Pointer Types                       │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   *T          Single-item pointer                        │
│               Points to exactly one T                    │
│               Example: *i32, *bool, *MyStruct            │
│                                                          │
│   [*]T        Many-item pointer                          │
│               Points to unknown number of T              │
│               No length information                      │
│                                                          │
│   []T         Slice (pointer + length)                   │
│               Pointer with runtime length                │
│               Safer than [*]T                            │
│                                                          │
│   *const T    Pointer to constant                        │
│               Cannot modify the value through this pointer │
│                                                          │
│   ?*T         Optional pointer                           │
│               Can be null                                │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Passing by Pointer

```zig
const std = @import("std");

// Takes a pointer - can modify the original value
fn increment(ptr: *i32) void {
    ptr.* += 1;
}

// Takes a slice - can modify all elements
fn doubleAll(values: []i32) void {
    for (values) |*v| {
        v.* *= 2;
    }
}

// Swap two values using pointers
fn swap(a: *i32, b: *i32) void {
    const temp = a.*;
```

```zig
    a.* = b.*;
    b.* = temp;
}

pub fn main() void {
    std.debug.print("=== PASSING BY POINTER ===\n\n", .{});

    var num: i32 = 5;
    std.debug.print("Initial: {d}\n", .{num});

    increment(&num);
    std.debug.print("After increment: {d}\n", .{num});

    increment(&num);
    increment(&num);
    std.debug.print("After 2 more: {d}\n", .{num});

    // Array modification
    var arr = [_]i32{ 1, 2, 3, 4, 5 };
    std.debug.print("\nArray before: {any}\n", .{arr});
    doubleAll(&arr);
    std.debug.print("Array after double: {any}\n", .{arr});

    // Swap
    var x: i32 = 100;
    var y: i32 = 200;
    std.debug.print("\nBefore swap: x={d}, y={d}\n", .{ x, y });
    swap(&x, &y);
    std.debug.print("After swap: x={d}, y={d}\n", .{ x, y });
}
```

Output:

```
=== PASSING BY POINTER ===

Initial: 5
After increment: 6
After 2 more: 8

Array before: { 1, 2, 3, 4, 5 }
Array after double: { 2, 4, 6, 8, 10 }

Before swap: x=100, y=200
After swap: x=200, y=100
```

Key Points

1. **& gets address** - Use &x to get a pointer to x
2. **`.*` dereferences** - Use `ptr.*` to access the value
3. **Pointers enable mutation** - Pass by pointer to modify original data
4. **Const pointers** - `*const T` prevents modification through that pointer
5. **No null by default** - Regular pointers can't be null; use ?*T for nullable

---

Chapter 8: Strings

In Zig, strings are just arrays of bytes (`[]const u8`). There's no special string type - this keeps the language simple and gives you full control.

String Types

```
┌──────────────────────────────────────────────────────────────────┐
│                    String Types in Zig                           │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│   Type                Description                                │
│   ────                ───────────                                │
│   []const u8          Slice of bytes (most common)              │
│   *const [N:0]u8      Pointer to null-terminated string         │
│   [:0]const u8        Sentinel-terminated slice                 │
│   [N]u8               Fixed-size byte array                     │
│                                                                  │
│   String literal "Hello":                                        │
│                                                                  │
│   Memory: │ H │ e │ l │ l │ o │ \0 │                            │
│           └───────────────────────┘                              │
│           Type: *const [5:0]u8                                   │
│           Coerces to: []const u8                                 │
│                                                                  │
│   Key insight: Strings are NOT special in Zig.                   │
│   They're just arrays of u8 (bytes).                             │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== STRING BASICS ===\n\n", .{});

    // String literal - type is *const [11:0]u8
    // But usually coerces to []const u8
    const greeting: []const u8 = "Hello, Zig!";

    std.debug.print("String: {s}\n", .{greeting});
    std.debug.print("Length: {d}\n", .{greeting.len});

    // Access individual bytes
    std.debug.print("First byte: '{c}' (ASCII {d})\n", .{ greeting[0], greeting[0] });
    std.debug.print("Last byte: '{c}'\n", .{greeting[greeting.len - 1]});

    // Iterate over characters
    std.debug.print("\nCharacters: ", .{});
    for (greeting) |char| {
        std.debug.print("{c} ", .{char});
    }
    std.debug.print("\n", .{});
```

```
}
```

Output:

```
=== STRING BASICS ===

String: Hello, Zig!
Length: 11
First byte: 'H' (ASCII 72)
Last byte: '!'

Characters: H e l l o ,   Z i g !
```

String Operations

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== STRING OPERATIONS ===\n\n", .{});

    const text = "The quick brown fox jumps";

    // Slicing (substrings)
    const first_word = text[0..3];
    const last_word = text[20..];

    std.debug.print("Original: {s}\n", .{text});
    std.debug.print("First word: {s}\n", .{first_word});
    std.debug.print("Last word: {s}\n", .{last_word});

    // Comparison
    std.debug.print("\n=== COMPARISON ===\n", .{});
    const a = "apple";
    const b = "apple";
    const c = "banana";

    std.debug.print("'{s}' == '{s}': {}\n", .{ a, b, std.mem.eql(u8, a, b) });
    std.debug.print("'{s}' == '{s}': {}\n", .{ a, c, std.mem.eql(u8, a, c) });

    // Finding substrings
    std.debug.print("\n=== SEARCHING ===\n", .{});
    if (std.mem.indexOf(u8, text, "quick")) |index| {
        std.debug.print("'quick' found at index {d}\n", .{index});
    }
    if (std.mem.indexOf(u8, text, "slow")) |index| {
        std.debug.print("'slow' found at index {d}\n", .{index});
    } else {
        std.debug.print("'slow' not found\n", .{});
    }

    // Prefix and suffix
    std.debug.print("\n=== PREFIX/SUFFIX ===\n", .{});
    std.debug.print("Starts with 'The': {}\n", .{std.mem.startsWith(u8, text, "The")});
```

```
    std.debug.print("Ends with 'jumps': {}\n", .{std.mem.endsWith(u8, text, "jumps")});
}
```

Output:

```
=== STRING OPERATIONS ===

Original: The quick brown fox jumps
First word: The
Last word: jumps

=== COMPARISON ===
'apple' == 'apple': true
'apple' == 'banana': false

=== SEARCHING ===
'quick' found at index 4
'slow' not found

=== PREFIX/SUFFIX ===
Starts with 'The': true
Ends with 'jumps': true
```

Multiline and Escape Sequences

```
const std = @import("std");

pub fn main() void {
    // Multiline strings with \\
    const poem =
        \\Roses are red,
        \\Violets are blue,
        \\Zig is awesome,
        \\And so are you!
    ;

    std.debug.print("=== MULTILINE STRING ===\n", .{});
    std.debug.print("{s}\n", .{poem});

    // Escape sequences
    std.debug.print("\n=== ESCAPE SEQUENCES ===\n", .{});
    const escaped = "Tab:\tNewline:\nBackslash:\\Quote:\"";
    std.debug.print("{s}\n", .{escaped});

    // Compile-time concatenation
    std.debug.print("\n=== CONCATENATION ===\n", .{});
    const first = "Hello, ";
    const second = "World!";
    const combined = first ++ second;  // Compile-time only
    std.debug.print("{s}\n", .{combined});
}
```

Output:

```
=== MULTILINE STRING ===
Roses are red,
Violets are blue,
Zig is awesome,
And so are you!

=== ESCAPE SEQUENCES ===
Tab:    Newline:
Backslash:\Quote:"

=== CONCATENATION ===
Hello, World!
```

Key Points

1. **Strings are `[]const u8`** - Just byte slices, nothing special
2. **Use `std.mem` for operations** - `eql`, `indexOf`, `startsWith`, etc.
3. **`++` concatenation is compile-time only** - Can't concat at runtime
4. **UTF-8 by default** - Zig strings are UTF-8 encoded
5. **`{s}` format specifier** - Use for printing strings

Chapter 9: Structs

Structs let you group related data together into custom types. They're one of the most important features for organizing your code.

Struct Memory Layout

```
┌──────────────────────────────────────────────────────────────┐
│                    Struct Layout                             │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   const Point = struct {                                     │
│       x: i32,                                                │
│       y: i32,                                                │
│   };                                                         │
│                                                              │
│   Memory layout of Point{ .x = 10, .y = 20 }:               │
│                                                              │
│   Address:  0x100         0x104                              │
│             ┌──────────────┬──────────────┐                  │
│   Field:    │      x       │      y       │                  │
│   Value:    │      10      │      20      │                  │
│             └──────────────┴──────────────┘                  │
│             │← 4 bytes →│ │← 4 bytes →│                     │
│                                                              │
│   Total size: 8 bytes                                        │
│   Fields are laid out in declaration order                   │
│                                                              │
```

```zig
const std = @import("std");

// Define a struct type
const Point = struct {
    x: i32,
    y: i32,
};

// Struct with default values
const Rectangle = struct {
    x: i32 = 0,
    y: i32 = 0,
    width: u32 = 100,
    height: u32 = 100,
};

pub fn main() void {
    std.debug.print("=== CREATING STRUCTS ===\n\n", .{});

    // Create with all fields
    const p1 = Point{ .x = 10, .y = 20 };
    std.debug.print("p1: ({d}, {d})\n", .{ p1.x, p1.y });

    // Create with defaults
    const r1 = Rectangle{};  // All defaults
    std.debug.print("r1: pos=({d},{d}) size={d}x{d}\n", .{ r1.x, r1.y, r1.width, r1.height });

    // Override some defaults
    const r2 = Rectangle{ .x = 50, .y = 50, .width = 200 };
    std.debug.print("r2: pos=({d},{d}) size={d}x{d}\n", .{ r2.x, r2.y, r2.width, r2.height });

    // Mutable struct
    var p2 = Point{ .x = 0, .y = 0 };
    p2.x = 100;
    p2.y = 200;
    std.debug.print("p2 (modified): ({d}, {d})\n", .{ p2.x, p2.y });

    // Size information
    std.debug.print("\nSize of Point: {d} bytes\n", .{@sizeOf(Point)});
    std.debug.print("Size of Rectangle: {d} bytes\n", .{@sizeOf(Rectangle)});
}
```

Output:

```
=== CREATING STRUCTS ===

p1: (10, 20)
r1: pos=(0,0) size=100x100
r2: pos=(50,50) size=200x100
p2 (modified): (100, 200)

Size of Point: 8 bytes
Size of Rectangle: 16 bytes
```

Struct Methods

```zig
const std = @import("std");

const Circle = struct {
    x: f64,
    y: f64,
    radius: f64,

    // Method - takes self as parameter
    pub fn area(self: Circle) f64 {
        return std.math.pi * self.radius * self.radius;
    }

    // Method - takes pointer, can modify
    pub fn scale(self: *Circle, factor: f64) void {
        self.radius *= factor;
    }

    // Method - compute distance to another circle
    pub fn distanceTo(self: Circle, other: Circle) f64 {
        const dx = self.x - other.x;
        const dy = self.y - other.y;
        return @sqrt(dx * dx + dy * dy);
    }

    // Associated function - no self parameter
    pub fn unit() Circle {
        return Circle{ .x = 0, .y = 0, .radius = 1 };
    }
};

pub fn main() void {
    std.debug.print("=== STRUCT METHODS ===\n\n", .{});

    var c1 = Circle{ .x = 0, .y = 0, .radius = 5 };

    std.debug.print("Circle: center=({d},{d}), radius={d}\n", .{ c1.x, c1.y, c1.radius });
    std.debug.print("Area: {d:.2}\n", .{c1.area()});

    // Modify with method
    c1.scale(2);
    std.debug.print("After scale(2): radius={d}, area={d:.2}\n", .{ c1.radius, c1.area() });

    // Distance between circles
    const c2 = Circle{ .x = 3, .y = 4, .radius = 1 };
    std.debug.print("\nDistance between circles: {d}\n", .{c1.distanceTo(c2)});

    // Associated function (like static method)
    const unit = Circle.unit();
    std.debug.print("Unit circle radius: {d}\n", .{unit.radius});
}
```

Output:

```
=== STRUCT METHODS ===

Circle: center=(0,0), radius=5
Area: 78.54

After scale(2): radius=10, area=314.16

Distance between circles: 5
Unit circle radius: 1
```

Key Points

1. **Named fields** - Access with `.fieldname`
2. **Default values** - Fields can have defaults
3. **Methods** - Functions with `self` parameter
4. **Associated functions** - Functions without `self` (like static)
5. **Pointer methods** - Use `*Self` to modify the struct

Chapter 10: Enums

Enums define a type with a fixed set of named values. They're perfect for representing choices or states.

Enum Basics

```
┌──────────────────────────────────────────────────────────────┐
│                      Enum Concept                            │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   const Direction = enum { north, south, east, west };       │
│                                                              │
│   Instead of using magic numbers:                            │
│       const NORTH = 0;  // What is 0?                        │
│       const SOUTH = 1;  // Easy to mix up                    │
│                                                              │
│   Use meaningful names:                                      │
│       const dir = Direction.north;  // Clear!                │
│                                                              │
│   Benefits:                                                  │
│   • Self-documenting code                                    │
│   • Compile-time checked                                     │
│   • Switch must handle all cases                             │
│   • Can have methods                                         │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

const Color = enum {
    red,
    green,
```

```
    blue,
    yellow,
    purple,
};

const Direction = enum {
    north,
    south,
    east,
    west,

    // Enum can have methods!
    pub fn opposite(self: Direction) Direction {
        return switch (self) {
            .north => .south,
            .south => .north,
            .east => .west,
            .west => .east,
        };
    }

    pub fn isVertical(self: Direction) bool {
        return self == .north or self == .south;
    }
};

pub fn main() void {
    std.debug.print("=== ENUM BASICS ===\n\n", .{});

    // Create enum values
    const favorite: Color = .blue;
    var current: Color = .red;

    std.debug.print("Favorite: {}\n", .{favorite});
    std.debug.print("Current: {}\n", .{current});

    // Change value
    current = .green;
    std.debug.print("Changed to: {}\n", .{current});

    // Comparison
    if (favorite == .blue) {
        std.debug.print("Blue is the favorite!\n", .{});
    }

    // Switch on enum (must handle all cases)
    std.debug.print("\n=== SWITCH ===\n", .{});
    const message = switch (current) {
        .red => "Stop!",
        .green => "Go!",
        .blue => "Cool",
        .yellow => "Caution",
        .purple => "Royal",
```

```
    };
    std.debug.print("Message for {}: {s}\n", .{ current, message });

    // Methods
    std.debug.print("\n=== METHODS ===\n", .{});
    const dir: Direction = .north;
    std.debug.print("Direction: {}\n", .{dir});
    std.debug.print("Opposite: {}\n", .{dir.opposite()});
    std.debug.print("Is vertical: {}\n", .{dir.isVertical()});
}
```

Output:

```
=== ENUM BASICS ===

Favorite: Color.blue
Current: Color.red
Changed to: Color.green

=== SWITCH ===
Message for Color.green: Go!

=== METHODS ===
Direction: Direction.north
Opposite: Direction.south
Is vertical: true
```

Key Points

1. **Exhaustive switch** - Must handle every enum value
2. **Methods on enums** - Enums can have their own methods
3. **Short syntax** - Use `.value` instead of `EnumType.value`
4. **Type safety** - Can't mix different enum types

Chapter 11: Unions

Unions can hold one of several types, but only one at a time. Tagged unions know which variant is active.

Tagged Union Concept

```
┌──────────────────────────────────────────────────────────────┐
│                      Tagged Union                            │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   const Value = union(enum) {                                │
│       int: i64,                                              │
│       float: f64,                                            │
│       text: []const u8,                                      │
│   };                                                         │
│                                                              │
```

```
│   Only ONE variant is active at a time:                          │
│                                                                   │
│    ┌──────────────────────────────────────┐                       │
│    │ Tag │            Payload             │                       │
│    ├─────┼────────────────────────────────┤                       │
│    │ int │ 42 (i64)                       │   ← Active            │
│    ├─────┼────────────────────────────────┤                       │
│    │float│ ──────────── (unused)          │                       │
│    ├─────┼────────────────────────────────┤                       │
│    │text │ ──────────── (unused)          │                       │
│    └─────┴────────────────────────────────┘                       │
│                                                                   │
│   The tag tells us which variant is currently stored.             │
│   Size = max(payload sizes) + tag size                            │
│                                                                   │
└───────────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

const Value = union(enum) {
    int: i64,
    float: f64,
    text: []const u8,
    none: void,

    pub fn describe(self: Value) void {
        switch (self) {
            .int => |n| std.debug.print("Integer: {d}\n", .{n}),
            .float => |f| std.debug.print("Float: {d:.2}\n", .{f}),
            .text => |s| std.debug.print("Text: {s}\n", .{s}),
            .none => std.debug.print("None\n", .{}),
        }
    }

    pub fn isNumeric(self: Value) bool {
        return switch (self) {
            .int, .float => true,
            .text, .none => false,
        };
    }
};

pub fn main() void {
    std.debug.print("=== TAGGED UNIONS ===\n\n", .{});

    // Create different variants
    const values = [_]Value{
        .{ .int = 42 },
        .{ .float = 3.14159 },
        .{ .text = "Hello" },
        .none,
    };
```

```zig
    for (values) |v| {
        std.debug.print("Is numeric: {}, ", .{v.isNumeric()});
        v.describe();
    }

    // Check active variant
    std.debug.print("\n=== CHECKING VARIANT ===\n", .{});
    const val: Value = .{ .int = 100 };

    if (val == .int) {
        std.debug.print("It's an integer!\n", .{});
    }

    // Switch with capture
    switch (val) {
        .int => |n| std.debug.print("Value is: {d}\n", .{n}),
        else => std.debug.print("Not an integer\n", .{}),
    }
}
```

Output:

```
=== TAGGED UNIONS ===

Is numeric: true, Integer: 42
Is numeric: true, Float: 3.14
Is numeric: false, Text: Hello
Is numeric: false, None

=== CHECKING VARIANT ===
It's an integer!
Value is: 100
```

Key Points

1. **One variant at a time** - Only one type is active
2. **Tag tracks active variant** - `union(enum)` adds automatic tag
3. **Switch with capture** - Extract the value with `|value|`
4. **Memory efficient** - Size is max payload + tag, not sum of all

---

Chapter 12: Control Flow

Zig provides standard control flow: `if`, `switch`, `while`, and `for`. Each can be used as an expression.

If Expression

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== IF EXPRESSIONS ===\n\n", .{});
```

```zig
    const x: i32 = 42;

    // Basic if statement
    if (x > 0) {
        std.debug.print("x is positive\n", .{});
    }

    // If-else
    if (x % 2 == 0) {
        std.debug.print("x is even\n", .{});
    } else {
        std.debug.print("x is odd\n", .{});
    }

    // If-else chain
    const score: u32 = 85;
    const grade = if (score >= 90)
        "A"
    else if (score >= 80)
        "B"
    else if (score >= 70)
        "C"
    else
        "F";

    std.debug.print("Score {d} = Grade {s}\n", .{ score, grade });

    // If as expression (ternary-like)
    const abs_x = if (x >= 0) x else -x;
    std.debug.print("Absolute value of {d}: {d}\n", .{ x, abs_x });

    const sign = if (x > 0) "positive" else if (x < 0) "negative" else "zero";
    std.debug.print("{d} is {s}\n", .{ x, sign });
}
```

Output:

```
=== IF EXPRESSIONS ===

x is positive
x is even
Score 85 = Grade B
Absolute value of 42: 42
42 is positive
```

Switch Expression

```
┌──────────────────────────────────────────────────────────────┐
│                      Switch Features                         │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│    • EXHAUSTIVE: Must handle all possible values             │
│    • EXPRESSIONS: Returns a value                            │
│    • RANGES: Match a range with ...                          │
```

```
│   • MULTIPLE: Match multiple values with ,                    │
│   • CAPTURE: Capture the value with |x|                       │
│                                                               │
│   switch (value) {                                            │
│       1 => "one",           // Single value                   │
│       2, 3 => "two or three", // Multiple values              │
│       4...10 => "four to ten", // Range                       │
│       else => "other",       // Catch-all                     │
│   }                                                           │
│                                                               │
└───────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

pub fn main() void {
    std.debug.print("=== SWITCH ===\n\n", .{});

    const day: u8 = 3;
    const name = switch (day) {
        1 => "Monday",
        2 => "Tuesday",
        3 => "Wednesday",
        4 => "Thursday",
        5 => "Friday",
        6, 7 => "Weekend",
        else => "Invalid",
    };
    std.debug.print("Day {d} is {s}\n", .{ day, name });

    // Range in switch
    const age: u32 = 25;
    const category = switch (age) {
        0...12 => "Child",
        13...19 => "Teenager",
        20...64 => "Adult",
        65...120 => "Senior",
        else => "Invalid age",
    };
    std.debug.print("Age {d}: {s}\n", .{ age, category });
}
```

Output:

```
=== SWITCH ===

Day 3 is Wednesday
Age 25: Adult
```

Loops

```zig
const std = @import("std");

pub fn main() void {
    // ========================================
```

```zig
    // WHILE LOOP
    // ========================================
    std.debug.print("=== WHILE LOOP ===\n", .{});

    var i: u32 = 0;
    while (i < 5) {
        std.debug.print("{d} ", .{i});
        i += 1;
    }
    std.debug.print("\n", .{});

    // While with continue expression
    var j: u32 = 0;
    while (j < 10) : (j += 1) {
        if (j % 2 != 0) continue;
        std.debug.print("{d} ", .{j});
    }
    std.debug.print("\n", .{});

    // ========================================
    // FOR LOOP
    // ========================================
    std.debug.print("\n=== FOR LOOP ===\n", .{});

    const numbers = [_]i32{ 10, 20, 30, 40, 50 };

    // Simple iteration
    for (numbers) |n| {
        std.debug.print("{d} ", .{n});
    }
    std.debug.print("\n", .{});

    // With index
    for (numbers, 0..) |n, idx| {
        std.debug.print("[{d}]={d} ", .{ idx, n });
    }
    std.debug.print("\n", .{});

    // Range iteration
    std.debug.print("Range: ", .{});
    for (0..5) |i2| {
        std.debug.print("{d} ", .{i2});
    }
    std.debug.print("\n", .{});
}
```

Output:

```
=== WHILE LOOP ===
0 1 2 3 4
0 2 4 6 8

=== FOR LOOP ===
10 20 30 40 50
```

```
[0]=10 [1]=20 [2]=30 [3]=40 [4]=50
Range: 0 1 2 3 4
```

Key Points

1. **Expressions, not statements** - Control flow can return values
2. **Exhaustive switch** - Must handle all cases
3. **No ++ operator** - Use += 1 or continue expression
4. **For captures value** - `|value|` or `|value, index|`
5. **Labeled blocks** - Use `blk: { break :blk value; }` for complex expressions

Chapter 13: Functions

Functions are reusable blocks of code. Zig functions can have compile-time parameters, making them very flexible.

Function Anatomy

```
┌──────────────────────────────────────────────────────────┐
│                    Function Anatomy                      │
├──────────────────────────────────────────────────────────┤
│                                                          │
│   fn functionName(param1: Type1, param2: Type2) ReturnType │
│   │       │            │                          │      │
│   │       │            └── Parameters ────────────┘      │
│   │       │                                              │
│   │       └── Function name (camelCase)                  │
│   │                                                      │
│   └── fn keyword                                         │
│                                                          │
│   pub fn add(a: i32, b: i32) i32 {                       │
│       return a + b;                                      │
│   }                                                      │
│   │                       │                              │
│   └── pub = visible       └── Return statement           │
│       outside module                                     │
│                                                          │
│   Special return types:                                  │
│   • void     - No return value                           │
│   • !T       - Can return error OR T                     │
│   • ?T       - Can return null OR T                      │
│   • noreturn - Function never returns                    │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

// Simple function
fn add(a: i32, b: i32) i32 {
```

```zig
    return a + b;
}

// Void return - no return value
fn greet(name: []const u8) void {
    std.debug.print("Hello, {s}!\n", .{name});
}

// Multiple return values via struct
fn divmod(num: i32, den: i32) struct { quotient: i32, remainder: i32 } {
    return .{
        .quotient = @divTrunc(num, den),
        .remainder = @mod(num, den),
    };
}

// Recursive function
fn factorial(n: u64) u64 {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}

pub fn main() void {
    std.debug.print("=== FUNCTIONS ===\n\n", .{});

    // Simple calls
    const sum = add(5, 3);
    std.debug.print("5 + 3 = {d}\n", .{sum});

    greet("Zig");

    // Multiple return values
    const result = divmod(17, 5);
    std.debug.print("17 / 5 = {d} remainder {d}\n", .{ result.quotient, result.remainder });

    // Recursion
    std.debug.print("5! = {d}\n", .{factorial(5)});
    std.debug.print("10! = {d}\n", .{factorial(10)});
}
```

Output:

```
=== FUNCTIONS ===

5 + 3 = 8
Hello, Zig!
17 / 5 = 3 remainder 2
5! = 120
10! = 3628800
```

Key Points

1. **camelCase for functions** - Zig convention
2. **Explicit types** - All parameters must have types

3. **Single return** - Use structs for multiple values
4. **No overloading** - One function per name (use comptime for generic)

Chapter 14: Error Handling

Zig has built-in error handling with error unions. Errors are values, not exceptions.

Error Union Concept

```
┌──────────────────────────────────────────────────────────────┐
│                      Error Unions                            │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│   Regular function:  fn divide(a: i32, b: i32) i32           │
│   Error function:    fn divide(a: i32, b: i32) !i32          │
│                                                  │           │
│                      Can return error OR value ──┘           │
│                                                              │
│   Error union: error OR value                                │
│                                                              │
│    ┌──────────────────┬──────────────────┐                  │
│    │     Error        │     Value        │                  │
│    │  (if failed)     │  (if success)    │                  │
│    └──────────────────┴──────────────────┘                  │
│                                                              │
│   Handling:                                                  │
│   • try   - Propagate error to caller                        │
│   • catch - Handle error locally                             │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

// Define error set
const MathError = error{
    DivisionByZero,
    NegativeNumber,
};

// Function that can fail
fn divide(a: i32, b: i32) MathError!i32 {
    if (b == 0) return error.DivisionByZero;
    return @divTrunc(a, b);
}

fn sqrt(x: i32) MathError!i32 {
    if (x < 0) return error.NegativeNumber;
    var r: i32 = 0;
    while (r * r <= x) : (r += 1) {}
    return r - 1;
```

```zig
}

pub fn main() void {
    std.debug.print("=== ERROR HANDLING ===\n\n", .{});

    // Method 1: Check with if
    if (divide(10, 2)) |result| {
        std.debug.print("10 / 2 = {d}\n", .{result});
    } else |err| {
        std.debug.print("Error: {}\n", .{err});
    }

    // Method 2: catch with default
    const safe = divide(10, 0) catch 0;
    std.debug.print("10 / 0 (with default): {d}\n", .{safe});

    // Method 3: catch with handling
    const result = divide(10, 0) catch |err| blk: {
        std.debug.print("Caught: {}\n", .{err});
        break :blk -1;
    };
    std.debug.print("Result: {d}\n", .{result});

    // Method 4: catch unreachable (assert no error)
    const guaranteed = divide(20, 4) catch unreachable;
    std.debug.print("20 / 4 = {d}\n", .{guaranteed});
}
```

Output:

```
=== ERROR HANDLING ===

10 / 2 = 5
10 / 0 (with default): 0
Caught: error.DivisionByZero
Result: -1
20 / 4 = 5
```

Error Propagation with try

```zig
const std = @import("std");

const MathError = error{ DivisionByZero, NegativeNumber };

fn divide(a: i32, b: i32) MathError!i32 {
    if (b == 0) return error.DivisionByZero;
    return @divTrunc(a, b);
}

fn sqrt(x: i32) MathError!i32 {
    if (x < 0) return error.NegativeNumber;
    var r: i32 = 0;
    while (r * r <= x) : (r += 1) {}
```

```zig
    return r - 1;
}

// try propagates errors automatically
fn calculate(a: i32, b: i32) MathError!i32 {
    const quotient = try divide(a, b);  // Returns if error
    const root = try sqrt(quotient);     // Returns if error
    return root;
}

pub fn main() void {
    std.debug.print("=== TRY PROPAGATION ===\n\n", .{});

    if (calculate(100, 4)) |result| {
        std.debug.print("calculate(100, 4) = {d}\n", .{result});
    } else |err| {
        std.debug.print("Error: {}\n", .{err});
    }

    if (calculate(100, 0)) |result| {
        std.debug.print("calculate(100, 0) = {d}\n", .{result});
    } else |err| {
        std.debug.print("calculate(100, 0) error: {}\n", .{err});
    }
}
```

Output:

```
=== TRY PROPAGATION ===

calculate(100, 4) = 5
calculate(100, 0) error: error.DivisionByZero
```

Key Points

1. **Errors are values** - Not exceptions, normal return values
2. **`try` propagates** - Returns error immediately if one occurs
3. **`catch` handles** - Provide default or handle locally
4. **Custom error sets** - Define your own error types
5. **`errdefer`** - Cleanup only on error path

Chapter 15: Optionals

Optionals represent values that might not exist. They're safer than null pointers.

Optional Concept

```
┌──────────────────────────────────────────────────────────────┐
│                      Optionals                               │
├──────────────────────────────────────────────────────────────┤
│                                                              │
```

```
│   ?T means "maybe T" - either a T value OR null                │
│                                                                 │
│   const maybe: ?i32 = 42;    // Has value                       │
│   const empty: ?i32 = null;  // No value                        │
│                                                                 │
│    ┌──────────────────────────────────────┐                     │
│    │  ?i32 with value                     │                     │
│    │  ┌──────────┬──────────────┐         │                     │
│    │  │ has_val  │    value     │         │                     │
│    │  │  true    │     42       │         │                     │
│    │  └──────────┴──────────────┘         │                     │
│    └──────────────────────────────────────┘                     │
│                                                                 │
│    ┌──────────────────────────────────────┐                     │
│    │  ?i32 without value (null)           │                     │
│    │  ┌──────────┬──────────────┐         │                     │
│    │  │ has_val  │    value     │         │                     │
│    │  │  false   │ (undefined)  │         │                     │
│    │  └──────────┴──────────────┘         │                     │
│    └──────────────────────────────────────┘                     │
│                                                                 │
│   Unwrapping:                                                   │
│   • if (opt) |val| { }    - Safe unwrap                         │
│   • opt orelse default    - Provide default                     │
│   • opt.?                 - Assert not null (crashes if is)     │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

```zig
const std = @import("std");

// Function returning optional
fn find(haystack: []const u8, needle: u8) ?usize {
    for (haystack, 0..) |byte, index| {
        if (byte == needle) return index;
    }
    return null;
}

fn getElement(arr: []const i32, index: usize) ?i32 {
    if (index >= arr.len) return null;
    return arr[index];
}

pub fn main() void {
    std.debug.print("=== OPTIONALS ===\n\n", .{});

    // Creating optionals
    var maybe: ?i32 = 42;
    std.debug.print("maybe = {?d}\n", .{maybe});

    maybe = null;
    std.debug.print("maybe (null) = {?d}\n", .{maybe});
```

```zig
    // If-unwrap pattern
    const text = "Hello, World!";
    if (find(text, 'W')) |index| {
        std.debug.print("'W' found at index {d}\n", .{index});
    }

    if (find(text, 'Z')) |index| {
        std.debug.print("'Z' found at index {d}\n", .{index});
    } else {
        std.debug.print("'Z' not found\n", .{});
    }

    // orelse - provide default
    const val1: ?i32 = 42;
    const val2: ?i32 = null;

    std.debug.print("\nval1 orelse 0: {d}\n", .{val1 orelse 0});
    std.debug.print("val2 orelse -1: {d}\n", .{val2 orelse -1});

    // Safe array access
    const numbers = [_]i32{ 10, 20, 30 };
    std.debug.print("\nnumbers[1] = {?d}\n", .{getElement(&numbers, 1)});
    std.debug.print("numbers[10] = {?d}\n", .{getElement(&numbers, 10)});
}
```

Output:

```
=== OPTIONALS ===

maybe = 42
maybe (null) = null
'W' found at index 7
'Z' not found

val1 orelse 0: 42
val2 orelse -1: -1

numbers[1] = 20
numbers[10] = null
```

Key Points

1. **`?T` = maybe T** - Value or null
2. **`if (opt) |val|`** - Safe unwrap with payload capture
3. **`orelse`** - Provide default if null
4. **`.?` operator** - Unwrap (panics if null)
5. **No null pointer exceptions** - Must handle null explicitly

---

Chapter 16: Comptime

Comptime runs code at compile time. This enables powerful metaprogramming without macros.

```zig
const std = @import("std");

// Compile-time constants
const KILOBYTE = 1024;
const MEGABYTE = KILOBYTE * 1024;
const GIGABYTE = MEGABYTE * 1024;

// Compile-time function
fn factorial(n: u64) u64 {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}

// Pre-computed at compile time
const FACT_10 = factorial(10);

pub fn main() void {
    std.debug.print("=== COMPTIME ===\n\n", .{});

    std.debug.print("1 KB = {d} bytes\n", .{KILOBYTE});
    std.debug.print("1 MB = {d} bytes\n", .{MEGABYTE});
    std.debug.print("1 GB = {d} bytes\n", .{GIGABYTE});

    std.debug.print("\n10! = {d} (computed at compile time!)\n", .{FACT_10});

    // Comptime block
    const sum = comptime blk: {
        var total: u32 = 0;
        for (1..11) |i| {
            total += @as(u32, @intCast(i));
        }
        break :blk total;
    };
    std.debug.print("Sum 1-10 = {d} (compile time)\n", .{sum});

    // Type reflection
    std.debug.print("\nType info:\n", .{});
    std.debug.print("  i32 size: {d} bytes\n", .{@sizeOf(i32)});
    std.debug.print("  i64 size: {d} bytes\n", .{@sizeOf(i64)});
}
```

Output:

```
=== COMPTIME ===

1 KB = 1024 bytes
1 MB = 1048576 bytes
1 GB = 1073741824 bytes

10! = 3628800 (computed at compile time!)
Sum 1-10 = 55 (compile time)

Type info:
  i32 size: 4 bytes
```

```
i64 size: 8 bytes
```

Chapter 17: Generics

Generics let you write code that works with any type.

```zig
const std = @import("std");

// Generic function
fn max(comptime T: type, a: T, b: T) T {
    return if (a > b) a else b;
}

fn swap(comptime T: type, a: *T, b: *T) void {
    const temp = a.*;
    a.* = b.*;
    b.* = temp;
}

// Generic struct
fn Stack(comptime T: type) type {
    return struct {
        const Self = @This();
        items: [100]T = undefined,
        count: usize = 0,

        pub fn push(self: *Self, item: T) void {
            self.items[self.count] = item;
            self.count += 1;
        }

        pub fn pop(self: *Self) ?T {
            if (self.count == 0) return null;
            self.count -= 1;
            return self.items[self.count];
        }
    };
}

pub fn main() void {
    std.debug.print("=== GENERICS ===\n\n", .{});

    // Generic functions
    std.debug.print("max(i32, 5, 10) = {d}\n", .{max(i32, 5, 10)});
    std.debug.print("max(f64, 3.14, 2.71) = {d}\n", .{max(f64, 3.14, 2.71)});

    var x: i32 = 100;
    var y: i32 = 200;
    std.debug.print("\nBefore swap: x={d}, y={d}\n", .{ x, y });
    swap(i32, &x, &y);
    std.debug.print("After swap: x={d}, y={d}\n", .{ x, y });
```

```zig
    // Generic struct
    std.debug.print("\n=== GENERIC STACK ===\n", .{});
    var stack = Stack(i32){};
    stack.push(10);
    stack.push(20);
    stack.push(30);

    while (stack.pop()) |val| {
        std.debug.print("Popped: {d}\n", .{val});
    }
}
```

Output:

```
=== GENERICS ===

max(i32, 5, 10) = 10
max(f64, 3.14, 2.71) = 3.14

Before swap: x=100, y=200
After swap: x=200, y=100

=== GENERIC STACK ===
Popped: 30
Popped: 20
Popped: 10
```

Chapter 18: Memory Management

Zig gives you explicit control over memory allocation.

```zig
const std = @import("std");

pub fn main() !void {
    std.debug.print("=== MEMORY MANAGEMENT ===\n\n", .{});

    // Stack memory (automatic)
    var stack_var: i32 = 42;
    var stack_arr: [10]i32 = undefined;
    for (&stack_arr, 0..) |*item, i| {
        item.* = @as(i32, @intCast(i * 2));
    }
    std.debug.print("Stack var: {d}\n", .{stack_var});
    std.debug.print("Stack array: {any}\n", .{stack_arr});

    // Heap memory (manual)
    var gpa = std.heap.GeneralPurposeAllocator(.{}){};
    defer _ = gpa.deinit();
    const allocator = gpa.allocator();

    // Allocate single value
    const ptr = try allocator.create(i32);
```

```zig
    defer allocator.destroy(ptr);
    ptr.* = 100;
    std.debug.print("\nHeap value: {d}\n", .{ptr.*});

    // Allocate slice
    const slice = try allocator.alloc(i32, 5);
    defer allocator.free(slice);
    for (slice, 0..) |*item, i| {
        item.* = @as(i32, @intCast((i + 1) * 10));
    }
    std.debug.print("Heap slice: {any}\n", .{slice});

    // ArrayList
    var list = std.ArrayList(i32).init(allocator);
    defer list.deinit();
    try list.append(1);
    try list.append(2);
    try list.append(3);
    std.debug.print("ArrayList: {any}\n", .{list.items});
}
```

Output:

```
=== MEMORY MANAGEMENT ===

Stack var: 42
Stack array: { 0, 2, 4, 6, 8, 10, 12, 14, 16, 18 }

Heap value: 100
Heap slice: { 10, 20, 30, 40, 50 }
ArrayList: { 1, 2, 3 }
```

Chapter 19: Testing

Zig has built-in testing support.

```zig
const std = @import("std");
const expect = std.testing.expect;
const expectEqual = std.testing.expectEqual;

fn add(a: i32, b: i32) i32 {
    return a + b;
}

fn factorial(n: u32) u32 {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}

test "basic addition" {
    try expectEqual(@as(i32, 5), add(2, 3));
    try expectEqual(@as(i32, 0), add(-5, 5));
```

```zig
}

test "factorial" {
    try expectEqual(@as(u32, 1), factorial(0));
    try expectEqual(@as(u32, 1), factorial(1));
    try expectEqual(@as(u32, 120), factorial(5));
}

test "expect examples" {
    try expect(add(1, 1) == 2);
    try expect(10 > 5);
}

pub fn main() void {
    std.debug.print("Run tests with: zig test filename.zig\n", .{});
}
```

Run with: `zig test filename.zig`

Chapter 20: C Interoperability

Zig can call C code directly.

```zig
const std = @import("std");
const c = @cImport({
    @cInclude("math.h");
});

pub fn main() void {
    std.debug.print("=== C INTEROP ===\n\n", .{});

    // Call C math functions
    const x: c.double = 2.0;

    std.debug.print("sqrt({d}) = {d}\n", .{ x, c.sqrt(x) });
    std.debug.print("pow({d}, 3) = {d}\n", .{ x, c.pow(x, 3.0) });
    std.debug.print("sin({d}) = {d}\n", .{ x, c.sin(x) });
    std.debug.print("cos({d}) = {d}\n", .{ x, c.cos(x) });
}

// Export function for C to call
export fn zig_add(a: c_int, b: c_int) c_int {
    return a + b;
}
```

Output:

```
=== C INTEROP ===

sqrt(2) = 1.4142135623730951
pow(2, 3) = 8
sin(2) = 0.9092974268256817
cos(2) = -0.4161468365471424
```

Appendix A: Zig Cheat Sheet

```
┌──────────────────────────────────────────────────────────┐
│                      ZIG CHEAT SHEET                     │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  VARIABLES                                               │
│  ────────────────────────────────────────────────────    │
│  const x: i32 = 42;        // Immutable                  │
│  var y: i32 = 0;           // Mutable                    │
│  var z: i32 = undefined;   // Uninitialized              │
│                                                          │
│  TYPES                                                   │
│  ────────────────────────────────────────────────────    │
│  i8, i16, i32, i64, i128   // Signed integers            │
│  u8, u16, u32, u64, u128   // Unsigned integers          │
│  f16, f32, f64, f128       // Floats                     │
│  bool                      // Boolean                    │
│  []T                       // Slice                      │
│  [N]T                      // Array                      │
│  *T                        // Pointer                    │
│  ?T                        // Optional                   │
│  !T or E!T                 // Error union                │
│                                                          │
│  CONTROL FLOW                                            │
│  ────────────────────────────────────────────────────    │
│  if (cond) { } else { }                                  │
│  switch (x) { val => {}, else => {} }                    │
│  while (cond) : (cont) { }                               │
│  for (slice) |item| { }                                  │
│  for (slice, 0..) |item, i| { }                          │
│                                                          │
│  FUNCTIONS                                               │
│  ────────────────────────────────────────────────────    │
│  fn name(param: T) ReturnType { }                        │
│  fn name(param: T) !ReturnType { }  // Can error         │
│                                                          │
│  ERROR HANDLING                                          │
│  ────────────────────────────────────────────────────    │
│  try expression           // Propagate error             │
│  catch |err| { }          // Handle error                │
│  orelse default           // Unwrap optional             │
│  if (opt) |val| { }       // Unwrap optional             │
│                                                          │
│  MEMORY                                                  │
│  ────────────────────────────────────────────────────    │
│  allocator.create(T)      // Allocate one                │
```

```
│  allocator.alloc(T, n)    // Allocate many                    │
│  allocator.destroy(ptr)   // Free one                         │
│  allocator.free(slice)    // Free many                        │
│  defer cleanup();         // Run at scope exit                │
│                                                               │
│  COMMON BUILTINS                                              │
│  ─────────────────────────────────────────────────────────    │
│  @import("name")          // Import module                    │
│  @sizeOf(T)               // Size of type                     │
│  @typeInfo(T)             // Type information                 │
│  @intCast(val)            // Integer conversion               │
│  @floatCast(val)          // Float conversion                 │
│                                                               │
└───────────────────────────────────────────────────────────────┘
```

Appendix B: Common Patterns

```
┌───────────────────────────────────────────────────────────────┐
│                      COMMON PATTERNS                          │
├───────────────────────────────────────────────────────────────┤
│                                                               │
│  OPTION UNWRAPPING                                            │
│  ┌─────────────────────────────────────────────────────┐      │
│  │  if (optional) |value| {                            │      │
│  │      // use value                                   │      │
│  │  } else {                                           │      │
│  │      // handle null                                 │      │
│  │  }                                                  │      │
│  │                                                     │      │
│  │  const val = optional orelse default;               │      │
│  │  const val = optional orelse return error.NotFound;│     │
│  └─────────────────────────────────────────────────────┘      │
│                                                               │
│  ERROR HANDLING                                               │
│  ┌─────────────────────────────────────────────────────┐      │
│  │  const result = try riskyOperation();               │      │
│  │                                                     │      │
│  │  const result = riskyOperation() catch |err| {      │      │
│  │      log.err("Failed: {}", .{err});                 │      │
│  │      return err;                                    │      │
│  │  };                                                 │      │
│  │                                                     │      │
│  │  const result = riskyOperation() catch default;     │      │
│  └─────────────────────────────────────────────────────┘      │
│                                                               │
│  RESOURCE CLEANUP                                             │
│  ┌─────────────────────────────────────────────────────┐      │
│  │  const file = try fs.openFile(path);                │      │
```

```
│   │  defer file.close();                                  │    │
│   │  // use file...                                       │    │
│   │  // close() called automatically!                     │    │
│   │                                                       │    │
│   │  const ptr = try allocator.create(T);                 │    │
│   │  errdefer allocator.destroy(ptr);  // Only on error│    │
│   │  try initializePtr(ptr);                              │    │
│   └───────────────────────────────────────────────────────┘    │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

About the Author

JW Lee is a software developer passionate about systems programming and making complex topics accessible to beginners. This book represents his effort to create the most comprehensive yet accessible introduction to the Zig programming language.

Easy Zig - A Comprehensive Beginner's Guide to the Zig Programming Language

Copyright 2024 JW Lee

All examples in this book are provided under the MIT License.